PhenObs Cluster 1 analyses

Alicia Valdés

09 febrero, 2026

Contents

Load libraries needed **3**

Read data **4**

Resolve species names **7**

Species in short PhenObs species list (species with sensitivity data) . . . 7

Species in sensitivity data . . . 8

Flowering . . . 8

GSL . . . 8

Species in full PhenObs species list . . . 9

Species in Red List data . . . 9

Species in stress tolerance scores . . . 11

Species in Ellenberg tendency of change data . . . 11

Create occurrence matrix **13**

Get countries form RL . . . 13

Build occurrence matrix . . . 16

Matching Phenobs - MOTIVATE Red List data **18**

Add ISO2 country codes to RL . . . 19

Join RL and full PhenObs spp list . . . 19

Join RL and sensitivity PhenObs spp list . . . 19

Data prep RL . . . 20

Data prep RL for Germany . . . 22

Join occurrence data and Red List data **23**

Europe . . . 23

Germany . . . 23

Calculate % endangerment **24**

Data prep sensitivity data **25**

Flowering . . . 25

GSL . . . 26

Prepare datasets for models **27**

Europe . . . 27

Sensitivity + % endangerment . . . 27

Sensitivity + % endangerment + stress . . . 27

Sensitivity + % endangerment + Ellenberg tendency of change . . . 27

Germany: sensitivity + threat . . . 28

Distributions **29**

Models endangerment ~ sensitivity **37**

Europe . . . 37

Sensitive or not . . . 37

Mean slopes . . . 38

Flowering (absolute value) . . . 38

GSL (quadratic model) . . . 42

Germany (all NS) . . . 44

Sensitive or not . . . 44

Mean slopes . . . 45

Flowering (absolute value) . . . 45

GSL (quadratic model) . . . 46

Correlation mean slope flowering - GSL **47**

Models stress tolerance **49**

Endangerment ~ stress tolerance . . . 49

Endangerment ~ stress tolerance * sensitivity . . . 50

Sensitive or not . . . 50

Mean slopes . . . 51

Flowering (absolute) . . . 51

GSL (quadratic model) . . . 54

Models Ellenberg tendency of change ~ sensitivity **55**
Sensitive or not . . . 55
Mean slopes . . . 56
Flowering (absolute) . . . 56
All slopes . . . 56
Negative slopes only . . . 57
GSL (quadratic model) . . . 58

Session info **59**

Load libraries needed

```
library(readr)
library(here)
```

```
## here() starts at C:/Users/jimenezalfaro/OneDrive - Universidad de Oviedo/IMIB/Analyses/PhenObs_Cluste
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v purrr     1.2.1
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.2     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(purrr)
library(sf)
```

```
## Linking to GEOS 3.13.1, GDAL 3.11.4, PROJ 9.7.0; sf_use_s2() is TRUE
```

```
library(glmmTMB)
library(ggeffects)
library(taxize)
library(rgbif)
library(readxl)
library(gridExtra)
```

```
##
## Adjuntando el paquete: 'gridExtra'
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```
library(furrr)
```

```
## Cargando paquete requerido: future
##
## Adjuntando el paquete: 'future'
##
## The following object is masked from 'package:taxize':
##
##     resolve
```

```
library(countrycode)
library(progress)
```

Read data

Read species list from PhenObs (these are the 177 species from which we initially had data on phenological sensitivity, i.e. slopes calculated by Robert).

```
species <- read_csv(here("data", "raw", "list_species_PhenObs.csv"))
```

```
## Rows: 177 Columns: 1
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): species
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
species
```

```
## # A tibble: 177 x 1
##    species
##    <chr>
##  1 Achillea millefolium
##  2 Aconitum lycoctonum
##  3 Acorus calamus
##  4 Actaea spicata
##  5 Adonis vernalis
##  6 Aegopodium podagraria
##  7 Agrimonia eupatoria
##  8 Ajuga pyramidalis
##  9 Alchemilla vulgaris
## 10 Alchemilla xanthochlora
## # i 167 more rows
```

Read the full PhenObs species list from 2022 data:

```
sp_list <- read_excel("data/raw/full_list_sp_phenobs_2022.xlsx") %>%
  rename(species = Species)
```

Load Red List data (MOTIVATE subset).

```
redlist <- read_tsv(here("data", "raw", "redlist_MOTIVATE_subset.csv"))
```

```
## Rows: 56038 Columns: 26
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (23): sPlot concept unified, GBIF concept unified, match_name, original_...
## dbl  (3): V1, accepted_plant_name_id, Publ_year
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
redlist
```

```
## # A tibble: 56,038 x 26
##    `sPlot concept unified` `GBIF concept unified` match_name               V1
##    <chr>                   <chr>                  <chr>                  <dbl>
##  1 Abelia                  Abelia                 Abelia                    NA
##  2 Abelmoschus esculentus  Abelmoschus esculentus Abelmoschus esculentus    NA
##  3 Abies                   Abies                  Abies                     NA
##  4 Abies alba              Abies alba             Abies alba               686
##  5 Abies alba              Abies alba             Abies alba               687
##  6 Abies alba              Abies alba             Abies alba              5088
##  7 Abies alba              Abies alba             Abies alba              8698
##  8 Abies alba              Abies alba             Abies alba             11749
##  9 Abies alba              Abies alba             Abies alba             19497
## 10 Abies alba              Abies alba             Abies alba             23584
## # i 56,028 more rows
## # i 22 more variables: original_species_name <chr>, parsed_species_name <chr>,
## #   accepted_taxon_name <chr>, accepted_taxon_authors <chr>,
## #   original_RL_cat <chr>, Redlist_cat <chr>, Threat <chr>, Country <chr>,
## #   family <chr>, genus <chr>, taxon_rank <chr>, lifeform_description <chr>,
## #   match_type <chr>, match_status <chr>, accepted_plant_name_id <dbl>,
## #   ipni_id <chr>, Publ_year <dbl>, region_name <chr>, ...
```

Read sensitivity data (slopes from Robert).

```
sensitivity_data_fl <- read_csv(here("data", "raw", "slopes_20251128", "Overview_Slopes_Flowering.csv"))
  select(-...1)
```

```
## New names:
## Rows: 171 Columns: 11
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (1): Species dbl (10): ...1, mean_slope, ci_lower, ci_upper, slope_var,
## n_gardens, temp_m...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

```
sensitivity_data_GSL <- read_csv(here("data", "raw", "slopes_20251128", "Overview_Slopes_GSL.csv")) %>%
  select(-...1)
```

```
## New names:
## Rows: 154 Columns: 11
## -- Column specification
## -------------------------------------------------------- Delimiter: "," chr
## (1): Species dbl (10): ...1, mean_slope, ci_lower, ci_upper, slope_var,
## n_gardens, temp_m...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
```

Read stress tolerance scores.

```
stress <- read_delim(here("data", "raw", "Stress_Tolerance.csv"), delim = ";") %>%
  rename(stress_tol = `Stress tolerator score_percentage`)
```

```
## Rows: 263 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ";"
## chr (1): Species
## dbl (1): Stress tolerator score_percentage
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
stress
```

```
## # A tibble: 263 x 2
##    Species               stress_tol
##    <chr>                      <dbl>
##  1 Achillea millefolium        44.4
##  2 Aconitum lycoctonum         13.4
##  3 Acorus calamus              23.8
##  4 Actaea spicata              11.1
##  5 Adonis amurensis            24.4
##  6 Adonis vernalis             41.5
##  7 Aegopodium podagraria        6.7
##  8 Agrimonia eupatoria         41.6
##  9 Agrimonia pilosa            53.7
## 10 Ajuga pyramidalis           34.1
## # i 253 more rows
```

Read Ellenberg tendency of change data.

```
ellenberg <- read_excel(here("data", "raw", "IndicatorValues_Ellenberg.xlsx"))
```

```
## Warning: Expecting numeric in Q1081 / R1081C17: got '?'
```

```
## Warning: Expecting numeric in Q1287 / R1287C17: got '?'
```

```
## Warning: Expecting numeric in Q2746 / R2746C17: got '?'
```

```
ellenberg
```

```
## # A tibble: 2,792 x 19
##    NR_E  Art    GATT_NAME OEK_L OEK_T OEK_K OEK_F OEK_R OEK_N OEK_S LF_LEB LF_B
##    <chr> <chr>  <chr>     <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr>  <chr>
##  1 101   Abies~ Abies     (3)   5     4     x     x     x     0     P      I
##  2 401   Acer ~ Acer      (5)   6     4     5     7     6     0     P      S
##  3 403   Acer ~ Acer      (6)   8     4     3     8     4     0     P      S
##  4 404   Acer ~ Acer      (5)   6     6     6     7     7     0     P      S
##  5 405   Acer ~ Acer      (5)   8     4     4     8     6     0     P      S
##  6 408   Acer ~ Acer      (4)   6     4     x     x     x     0     P      S
##  7 409   Acer ~ Acer      (4)   x     4     6     x     7     0     P      S
##  8 501   Acera~ Aceras    7     7     2     4~    8     3     0     G      S
##  9 601   Achil~ Achillea  9     2     4     5     8     3     0     H      W
## 10 623   Achil~ Achillea  8     6     6     8~    7     6     0     H      S
## # i 2,782 more rows
## # i 7 more variables: SZ_VERH_GR <chr>, SZ_VERH_K <chr>, SZ_O <dbl>,
## #   SZ_V <dbl>, HAEUF_M <dbl>, HAEUF_D <chr>, HAEUF_AE_change <dbl>
```

Resolve species names

Resolve species in all datasets to GBIF accepted names using the package taxize:

Species in short PhenObs species list (species with sensitivity data)

```
length(species$species) # 177
```

```
## [1] 177
```

```
# Due to gna_verifier only being able to handle 50 names at once subsetting
spp1 <- species$species[1:50]
spp2 <- species$species[51:100]
spp3 <- species$species[101:150]
spp4 <- species$species[151:177]

# Getting current canonical names
resolved_1 <- gna_verifier(spp1, data_sources = 11) # 11 = GBIF Backbone Taxonomy
resolved_2 <- gna_verifier(spp2, data_sources = 11)
resolved_3 <- gna_verifier(spp3, data_sources = 11)
resolved_4 <- gna_verifier(spp4, data_sources = 11)

# Combining all of the names
all_resolved <- rbind(resolved_1, resolved_2, resolved_3, resolved_4)

# Add to sp_list
species_resolved <- species %>%
  cbind(., all_resolved$currentCanonicalFull) %>%
  rename(name_res = `all_resolved$currentCanonicalFull`)
```

Species in sensitivity data

Flowering

```
length(sensitivity_data_fl$Species) # 171
```

```
## [1] 171
```

```
# Due to gna_verifier only being able to handle 50 names at once subsetting
spp1 <- sensitivity_data_fl$Species[1:50]
spp2 <- sensitivity_data_fl$Species[51:100]
spp3 <- sensitivity_data_fl$Species[101:150]
spp4 <- sensitivity_data_fl$Species[151:171]

# Unconventional way for getting current canonical names
resolved_1 <- gna_verifier(spp1, data_sources = 11) # 11 = GBIF Backbone Taxonomy
resolved_2 <- gna_verifier(spp2, data_sources = 11)
resolved_3 <- gna_verifier(spp3, data_sources = 11)
resolved_4 <- gna_verifier(spp4, data_sources = 11)

# Combining all of the names
all_resolved <- rbind(resolved_1, resolved_2, resolved_3, resolved_4)

# Add to sp_list
sensitivity_data_fl_resolved <- sensitivity_data_fl %>%
  cbind(., all_resolved$currentCanonicalFull) %>%
  rename(name_res = `all_resolved$currentCanonicalFull`)
```

GSL

```
length(sensitivity_data_GSL$Species) # 154
```

```
## [1] 154
```

```
# Due to gna_verifier only being able to handle 50 names at once subsetting
spp1 <- sensitivity_data_GSL$Species[1:50]
spp2 <- sensitivity_data_GSL$Species[51:100]
spp3 <- sensitivity_data_GSL$Species[101:150]
spp4 <- sensitivity_data_GSL$Species[151:154]

# Unconventional way for getting current canonical names
resolved_1 <- gna_verifier(spp1, data_sources = 11) # 11 = GBIF Backbone Taxonomy
resolved_2 <- gna_verifier(spp2, data_sources = 11)
resolved_3 <- gna_verifier(spp3, data_sources = 11)
resolved_4 <- gna_verifier(spp4, data_sources = 11)

# Combining all of the names
all_resolved <- rbind(resolved_1, resolved_2, resolved_3, resolved_4)
```

```
# Add to sp_list
sensitivity_data_GSL_resolved <- sensitivity_data_GSL %>%
  cbind(., all_resolved$currentCanonicalFull) %>%
  rename(name_res = `all_resolved$currentCanonicalFull`)
```

Species in full PhenObs species list

Resolve species to GBIF accepted names:

```
length(sp_list$species) # 416
```

```
## [1] 416
```

```
# Due to gna_verifier only being able to handle 50 names at once subsetting
spp1 <- sp_list$species[1:50]
spp2 <- sp_list$species[51:100]
spp3 <- sp_list$species[101:150]
spp4 <- sp_list$species[151:200]
spp5 <- sp_list$species[201:250]
spp6 <- sp_list$species[251:300]
spp7 <- sp_list$species[301:350]
spp8 <- sp_list$species[351:400]
spp9 <- sp_list$species[401:416]

# Unconventional way for getting current canonical names
resolved_1 <- gna_verifier(spp1, data_sources = 11) # 11 = GBIF Backbone Taxonomy
resolved_2 <- gna_verifier(spp2, data_sources = 11)
resolved_3 <- gna_verifier(spp3, data_sources = 11)
resolved_4 <- gna_verifier(spp4, data_sources = 11)
resolved_5 <- gna_verifier(spp5, data_sources = 11)
resolved_6 <- gna_verifier(spp6, data_sources = 11)
resolved_7 <- gna_verifier(spp7, data_sources = 11)
resolved_8 <- gna_verifier(spp8, data_sources = 11)
resolved_9 <- gna_verifier(spp9, data_sources = 11)


# Combining all of the names
all_resolved <- rbind(resolved_1, resolved_2, resolved_3, resolved_4,
                      resolved_5, resolved_6, resolved_7, resolved_8,
                      resolved_9)

# Add to sp_list
sp_list_resolved <- sp_list %>%
  cbind(., all_resolved$currentCanonicalFull) %>%
  rename(name_res = `all_resolved$currentCanonicalFull`)
```

Species in Red List data

Resolve species (`sPlot concept unified`) to GBIF accepted names:

```r
length(unique(redlist$`sPlot concept unified`)) # 21096
```

```
## [1] 21096
```

```r
# Due to gna_verifier only being able to handle 50 names at once subsetting
# Chunk the names into groups of <= 50
chunk_size <- 50
chunks <- split(unique(redlist$`sPlot concept unified`),
                ceiling(seq_along(unique(redlist$`sPlot concept unified`)) /
                          chunk_size))

# Helper to resolve one chunk using GBIF Backbone (data_sources = 11)
resolve_chunk_tbl <- function(x) {
  gna_verifier(
    names = x,
    data_sources = 11,        # GBIF Backbone Taxonomy
    all_matches = FALSE,      # best match only
    output_type = "table"
  )
}

# Try table output first
resolved_tbl <- tryCatch(
  {
    map(chunks, resolve_chunk_tbl) %>% bind_rows()
  },
  error = function(e) NULL
)
```

```r
resolved_tbl2 <- resolved_tbl %>%
  rename(
    rl_name_clean = submittedName,
    name_res = currentCanonicalFull
  ) %>%
  # If duplicates exist in resolved_tbl (same submitted name multiple times),
  # keep the first/best one:
  distinct(rl_name_clean, .keep_all = TRUE)

redlist_with_clean <- redlist %>%
  mutate(
    rl_name_raw   = `sPlot concept unified`,
    rl_name_clean = str_squish(rl_name_raw)
  )

# Left-join: keep ALL rows from redlist
redlist_resolved <- redlist_with_clean %>%
  left_join(resolved_tbl2 %>%
              select(rl_name_clean, name_res),
            by = "rl_name_clean")
```

Species in stress tolerance scores

```
length(stress$Species) # 263
```

```
## [1] 263
```

```
# Due to gna_verifier only being able to handle 50 names at once subsetting
spp1 <- stress$Species[1:50]
spp2 <- stress$Species[51:100]
spp3 <- stress$Species[101:150]
spp4 <- stress$Species[151:200]
spp5 <- stress$Species[201:250]
spp6 <- stress$Species[251:263]

# Unconventional way for getting current canonical names
resolved_1 <- gna_verifier(spp1, data_sources = 11) # 11 = GBIF Backbone Taxonomy
resolved_2 <- gna_verifier(spp2, data_sources = 11)
resolved_3 <- gna_verifier(spp3, data_sources = 11)
resolved_4 <- gna_verifier(spp4, data_sources = 11)
resolved_5 <- gna_verifier(spp5, data_sources = 11)
resolved_6 <- gna_verifier(spp6, data_sources = 11)

# Combining all of the names
all_resolved <- rbind(resolved_1, resolved_2, resolved_3, resolved_4,
                      resolved_5, resolved_6)

# Add to sp_list
stress_resolved <- stress %>%
  cbind(., all_resolved$currentCanonicalFull) %>%
  rename(name_res = `all_resolved$currentCanonicalFull`)
```

Species in Ellenberg tendency of change data

```
ellenberg <- ellenberg %>%
  mutate(Art = ifelse(Art == "Primula veris (officinalis)#f)",
                      "Primula veris (officinalis)", Art))
```

```
length(unique(ellenberg$Art)) # 2791
```

```
## [1] 2791
```

```
# Due to gna_verifier only being able to handle 50 names at once subsetting
# Chunk the names into groups of <= 50
chunks <- split(unique(ellenberg$Art),
                ceiling(seq_along(unique(ellenberg$Art)) /
                          chunk_size))
```

```r
resolved_tbl_list <- vector("list", length(chunks))
for (i in seq_along(chunks)) {
  cat("Resolving chunk", i, "of", length(chunks),
      " (n =", length(chunks[[i]]), ")\n")
  res <- try(resolve_chunk_tbl(chunks[[i]]), silent = TRUE)
  if (inherits(res, "try-error")) {
    cat("Chunk", i, "failed with:\n", attr(res, "condition")$message, "\n")
    break
  }
  resolved_tbl_list[[i]] <- res
}
```

```
## Resolving chunk 1 of 56  (n = 50 )
## Resolving chunk 2 of 56  (n = 50 )
## Resolving chunk 3 of 56  (n = 50 )
## Resolving chunk 4 of 56  (n = 50 )
## Resolving chunk 5 of 56  (n = 50 )
## Resolving chunk 6 of 56  (n = 50 )
## Resolving chunk 7 of 56  (n = 50 )
## Resolving chunk 8 of 56  (n = 50 )
## Resolving chunk 9 of 56  (n = 50 )
## Resolving chunk 10 of 56  (n = 50 )
## Resolving chunk 11 of 56  (n = 50 )
## Resolving chunk 12 of 56  (n = 50 )
## Resolving chunk 13 of 56  (n = 50 )
## Resolving chunk 14 of 56  (n = 50 )
## Resolving chunk 15 of 56  (n = 50 )
## Resolving chunk 16 of 56  (n = 50 )
## Resolving chunk 17 of 56  (n = 50 )
## Resolving chunk 18 of 56  (n = 50 )
## Resolving chunk 19 of 56  (n = 50 )
## Resolving chunk 20 of 56  (n = 50 )
## Resolving chunk 21 of 56  (n = 50 )
## Resolving chunk 22 of 56  (n = 50 )
## Resolving chunk 23 of 56  (n = 50 )
## Resolving chunk 24 of 56  (n = 50 )
## Resolving chunk 25 of 56  (n = 50 )
## Resolving chunk 26 of 56  (n = 50 )
## Resolving chunk 27 of 56  (n = 50 )
## Resolving chunk 28 of 56  (n = 50 )
## Resolving chunk 29 of 56  (n = 50 )
## Resolving chunk 30 of 56  (n = 50 )
## Resolving chunk 31 of 56  (n = 50 )
## Resolving chunk 32 of 56  (n = 50 )
## Resolving chunk 33 of 56  (n = 50 )
## Resolving chunk 34 of 56  (n = 50 )
## Resolving chunk 35 of 56  (n = 50 )
## Resolving chunk 36 of 56  (n = 50 )
## Resolving chunk 37 of 56  (n = 50 )
## Resolving chunk 38 of 56  (n = 50 )
## Resolving chunk 39 of 56  (n = 50 )
## Resolving chunk 40 of 56  (n = 50 )
## Resolving chunk 41 of 56  (n = 50 )
```

```
## Resolving chunk 42 of 56  (n = 50 )
## Resolving chunk 43 of 56  (n = 50 )
## Resolving chunk 44 of 56  (n = 50 )
## Resolving chunk 45 of 56  (n = 50 )
## Resolving chunk 46 of 56  (n = 50 )
## Resolving chunk 47 of 56  (n = 50 )
## Resolving chunk 48 of 56  (n = 50 )
## Resolving chunk 49 of 56  (n = 50 )
## Resolving chunk 50 of 56  (n = 50 )
## Resolving chunk 51 of 56  (n = 50 )
## Resolving chunk 52 of 56  (n = 50 )
## Resolving chunk 53 of 56  (n = 50 )
## Resolving chunk 54 of 56  (n = 50 )
## Resolving chunk 55 of 56  (n = 50 )
## Resolving chunk 56 of 56  (n = 41 )
```

```
resolved_tbl <- dplyr::bind_rows(resolved_tbl_list)
```

```
resolved_tbl2 <- resolved_tbl %>%
  rename(
    ellenberg_name_clean = submittedName,
    name_res = currentCanonicalFull
  ) %>%
  # If duplicates exist in resolved_tbl (same submitted name multiple times),
  # keep the first/best one:
  distinct(ellenberg_name_clean, .keep_all = TRUE)

ellenberg_with_clean <- ellenberg %>%
  mutate(
    ellenberg_name_raw   = Art,
    ellenberg_name_clean = str_squish(ellenberg_name_raw)
  )

# Left-join: keep ALL rows from ellenberg
ellenberg_resolved <- ellenberg_with_clean %>%
  left_join(resolved_tbl2 %>%
              select(ellenberg_name_clean, name_res),
            by = "ellenberg_name_clean") %>% select(-Art, -ellenberg_name_raw,
                                                    -ellenberg_name_clean)
```

Create occurrence matrix

Occurrence matrices are created using the package rgbif.

Get countries form RL

We want to get an occurrence matrix for all countries in Europe that are included in our Red List data.

```
# Inspect all countries present in Red List sample
redlist %>% distinct(Country) %>% arrange(Country) %>% print(n = 50)
```

```
## # A tibble: 43 x 1
##    Country
##    <chr>
##  1 Albania
##  2 Andorra
##  3 Austria
##  4 Belarus
##  5 Belgium
##  6 Bosnia and Herzegovina
##  7 Bulgaria
##  8 Croatia
##  9 Cyprus
## 10 Czech Republic
## 11 Denmark
## 12 Estonia
## 13 Finland
## 14 France
## 15 Germany
## 16 Greece
## 17 Hungary
## 18 Ireland
## 19 Italy
## 20 Kosovo
## 21 Latvia
## 22 Liechtenstein
## 23 Lithuania
## 24 Luxembourg
## 25 Malta
## 26 Moldova
## 27 Montenegro
## 28 Netherlands
## 29 Norway
## 30 Poland
## 31 Portugal
## 32 Romania
## 33 Russia
## 34 Serbia
## 35 Slovakia
## 36 Slovenia
## 37 Spain
## 38 Sweden
## 39 Switzerland
## 40 Turkey
## 41 Ukraine
## 42 United Kingdom
## 43 <NA>
```

Get vector of countries (remove NA).

```
countries_raw <- redlist %>%
  distinct(Country) %>%
  filter(!is.na(Country)) %>%
  pull(Country)
```

Convert country names to ISO2 country codes (used by rgbif).

```
# Custom mapping for tricky names
# - GBIF wants 'GB' for United Kingdom / Great Britain
# - 'Czech Republic' -> 'CZ'
# - 'Bosnia and Herzegovina' -> 'BA'
# - 'Russia' -> 'RU'
# - 'Kosovo' -> 'XK' (unofficial but accepted by GBIF)
custom_match <- c(
  "United Kingdom"        = "GB",
  "Great Britain"         = "GB",
  "Czech Republic"        = "CZ",
  "Bosnia and Herzegovina"= "BA",
  "Russia"                = "RU",
  "Kosovo"                = "XK"
)

# Convert to ISO-2 (countrycode handles the rest)
countries_iso <- tibble(
  country_name = countries_raw,
  iso2 = countrycode(countries_raw,
                     origin = "country.name",
                     destination = "iso2c",
                     custom_match = custom_match)
) %>%
  arrange(country_name)
countries_iso
```

```
## # A tibble: 42 x 2
##    country_name           iso2
##    <chr>                  <chr>
##  1 Albania                AL
##  2 Andorra                AD
##  3 Austria                AT
##  4 Belarus                BY
##  5 Belgium                BE
##  6 Bosnia and Herzegovina BA
##  7 Bulgaria               BG
##  8 Croatia                HR
##  9 Cyprus                 CY
## 10 Czech Republic         CZ
## # i 32 more rows
```

```
# Final clean vector for rgbif
gbif_iso2 <- countries_iso %>%
  filter(!is.na(iso2)) %>%
  distinct(iso2) %>%
  pull(iso2)

gbif_iso2
```

```
##  [1] "AL" "AD" "AT" "BY" "BE" "BA" "BG" "HR" "CY" "CZ" "DK" "EE" "FI" "FR" "DE"
## [16] "GR" "HU" "IE" "IT" "XK" "LV" "LI" "LT" "LU" "MT" "MD" "ME" "NL" "NO" "PL"
## [31] "PT" "RO" "RU" "RS" "SK" "SI" "ES" "SE" "CH" "TR" "UA" "GB"
```

Build occurrence matrix

Build the species × country occurrence matrix with rgbif.

Get species and countries vectors.

```
sp_vec <- unique(species_resolved$name_res)
cty_vec <- unique(gbif_iso2)
```

Use parallel GBIF querying.

```
options(timeout = 180)                          # allow slower responses
options(future.rng.onMisuse = "ignore")         # silence RNG warning when seed is set
```

```
`%||%` <- function(a, b) if (!is.null(a)) a else b

# Retry wrapper: tries 'expr_fun()' up to 'tries' times with exponential backoff
retry <- function(expr_fun, tries = 3L, base_wait = 0.7) {
  for (i in seq_len(tries)) {
    ok <- try(expr_fun(), silent = TRUE)
    if (!inherits(ok, "try-error")) return(ok)
    Sys.sleep(base_wait * (2^(i - 1)))  # 0.7s, 1.4s, 2.8s ...
  }
  ok
}

# Robust count helper using occ_count() with retries
get_species_country_count <- function(sp, iso2) {
  # Load packages inside each worker
  require(rgbif, quietly = TRUE, warn.conflicts = FALSE)
  require(tibble, quietly = TRUE, warn.conflicts = FALSE)

  out <- retry(function() rgbif::occ_count(scientificName = sp, country = iso2))

  # If still failed, coerce to 0
  if (inherits(out, "try-error") || is.null(out)) {
    return(tibble(species = sp, iso2 = iso2, count = 0L, status = "error"))
  }

  tibble(species = sp, iso2 = iso2, count = as.integer(out %||% 0L), status = "ok")
}
```

Build species × country combinations.

```
pairs_df <- tidyr::expand_grid(
  species = sp_vec,
  iso2    = gbif_iso2
)
```

```
workers <- 4
future::plan(multisession, workers = workers)

batch_size <- 100L
```

```r
borders <- seq(1L, nrow(pairs_df), by = batch_size)
batch_ids <- cut(seq_len(nrow(pairs_df)),
                 breaks = c(borders, nrow(pairs_df) + 1L),
                 include.lowest = TRUE, labels = FALSE)

batches <- split(pairs_df, batch_ids)

# Progress bar across batches (not inside workers)
pb <- progress_bar$new(
  format = "GBIF batches [:bar] :current/:total (:percent) eta: :eta",
  total = length(batches),
  clear = FALSE
)

all_results <- vector("list", length(batches))
error_log  <- list()

for (b in seq_along(batches)) {
  chunk <- batches[[b]]

  # Parallel over the rows of this chunk
  chunk_res <- try(
    {
      future_pmap(
        .l = chunk,  # pass the whole data frame (= list of cols)
        .f = function(species, iso2) {
          get_species_country_count(species, iso2)
        },
        .options = furrr_options(seed = TRUE, scheduling = 1) # scheduling=1 = predictable
      )
    },
    silent = TRUE
  )

  if (inherits(chunk_res, "try-error")) {
    # Record batch-level failure
    error_log[[length(error_log) + 1]] <- list(batch = b, error = as.character(chunk_res))
    # Fallback: try sequentially for this batch to salvage results
    chunk_res <- purrr::pmap(chunk, function(species, iso2) get_species_country_count(species, iso2))
  }

  # Bind rows from this batch
  all_results[[b]] <- dplyr::bind_rows(chunk_res)
  pb$tick()
}

results <- dplyr::bind_rows(all_results)

# Inspect failures, if any
fail_count <- sum(results$status != "ok")
if (fail_count > 0) {
  message("Requests with errors after retries: ", fail_count,
          ". They were set to count = 0. See 'error_log' for batch-level issues.")
```

```
}

plan(sequential)
```

Checks.

```
results %>% count(status)
```

```
## # A tibble: 1 x 2
##   status     n
##   <chr>  <int>
## 1 ok      7434
```

```
fail_count
```

```
## [1] 0
```

Build the presence/absence matrix.

```
occ_matrix_pa <- results %>%
  mutate(presence = as.integer(count > 0)) %>%
  select(species, iso2, presence) %>%
  pivot_wider(
    names_from  = iso2,
    values_from = presence,
    values_fill = 0
  ) %>%
  arrange(species) %>%
  # Add col with n countries present
  mutate(n_countries = rowSums(select(., -species)))
```

```
occ_matrix_pa %>% filter(n_countries == 0)
```

```
## # A tibble: 0 x 44
## # i 44 variables: species <chr>, AL <int>, AD <int>, AT <int>, BY <int>,
## #   BE <int>, BA <int>, BG <int>, HR <int>, CY <int>, CZ <int>, DK <int>,
## #   EE <int>, FI <int>, FR <int>, DE <int>, GR <int>, HU <int>, IE <int>,
## #   IT <int>, XK <int>, LV <int>, LI <int>, LT <int>, LU <int>, MT <int>,
## #   MD <int>, ME <int>, NL <int>, NO <int>, PL <int>, PT <int>, RO <int>,
## #   RU <int>, RS <int>, SK <int>, SI <int>, ES <int>, SE <int>, CH <int>,
## #   TR <int>, UA <int>, GB <int>, n_countries <dbl>
```

Matching Phenobs - MOTIVATE Red List data

Do not only use the RL of the countries where we have our botanical gardens, instead use the RL from all available countries in Europe!

Try to match the full PhenObs spp list with the RL to see how many species are there in each category for each country.

Add ISO2 country codes to RL

```
redlist_resolved <- redlist_resolved %>%
  left_join(countries_iso %>% rename(Country = country_name))
```

```
## Joining with `by = join_by(Country)`
```

Join RL and full PhenObs spp list

```
redlist_phenobs_all <- tibble(sp_list_resolved) %>%
  left_join(redlist_resolved) %>%
  # Keep columns that I think I will use
  select(name_res, Redlist_cat, Threat, Country, Publ_year, redlist_source,
         Continent, iso2)
```

```
## Joining with `by = join_by(name_res)`
```

```
redlist_phenobs_all
```

```
## # A tibble: 3,955 x 8
##    name_res  Redlist_cat Threat Country Publ_year redlist_source Continent iso2
##    <chr>     <chr>       <chr>  <chr>       <dbl> <chr>          <chr>     <chr>
##  1 Achillea~ LC          not_t~ Belgium      2019 rl_europe      Europe    BE
##  2 Achillea~ LC          not_t~ Estonia      2008 rl_NationalRL  Europe    EE
##  3 Achillea~ LC          not_t~ France       2018 rl_europe      Europe    FR
##  4 Achillea~ LC          not_t~ Germany      1999 rl_NationalRL  Europe    DE
##  5 Achillea~ LC          not_t~ Germany      2018 rl_europe      Europe    DE
##  6 Achillea~ LC          not_t~ Ireland      2016 rl_NationalRL  Europe    IE
##  7 Achillea~ LC          not_t~ Luxemb~      2005 rl_europe      Europe    LU
##  8 Achillea~ LC          not_t~ Nether~      2012 rl_NationalRL  Europe    NL
##  9 Achillea~ LC          not_t~ Switze~      2002 rl_NationalRL  Europe    CH
## 10 Achillea~ LC          not_t~ United~      2014 rl_NationalRL  Europe    GB
## # i 3,945 more rows
```

Join RL and sensitivity PhenObs spp list

```
redlist_phenobs <- tibble(species_resolved) %>%
  left_join(redlist_resolved) %>%
  # Keep columns that I think I will use
  select(name_res, Redlist_cat, Threat, Country, Publ_year, redlist_source,
         Continent, iso2)
```

```
## Joining with `by = join_by(name_res)`
```

```
redlist_phenobs
```

```
## # A tibble: 2,189 x 8
##    name_res  Redlist_cat Threat Country Publ_year redlist_source Continent iso2
##    <chr>     <chr>       <chr>  <chr>       <dbl> <chr>          <chr>     <chr>
##  1 Achillea~ LC          not_t~ Belgium      2019 rl_europe      Europe    BE
##  2 Achillea~ LC          not_t~ Estonia      2008 rl_NationalRL  Europe    EE
##  3 Achillea~ LC          not_t~ France       2018 rl_europe      Europe    FR
##  4 Achillea~ LC          not_t~ Germany      1999 rl_NationalRL  Europe    DE
##  5 Achillea~ LC          not_t~ Germany      2018 rl_europe      Europe    DE
##  6 Achillea~ LC          not_t~ Ireland      2016 rl_NationalRL  Europe    IE
##  7 Achillea~ LC          not_t~ Luxemb~      2005 rl_europe      Europe    LU
##  8 Achillea~ LC          not_t~ Nether~      2012 rl_NationalRL  Europe    NL
##  9 Achillea~ LC          not_t~ Switze~      2002 rl_NationalRL  Europe    CH
## 10 Achillea~ LC          not_t~ United~      2014 rl_NationalRL  Europe    GB
## # i 2,179 more rows
```

Data prep RL

Cases where there is info for more than one year for a species and country:

```
redlist_phenobs_dupl_diff <- redlist_phenobs %>%
  group_by(name_res, iso2) %>%
  filter(n() > 1, n_distinct(Threat) > 1) %>%
  ungroup()
redlist_phenobs_dupl_diff
```

```
## # A tibble: 809 x 8
##    name_res  Redlist_cat Threat Country Publ_year redlist_source Continent iso2
##    <chr>     <chr>       <chr>  <chr>       <dbl> <chr>          <chr>     <chr>
##  1 Acorus c~ VU          threa~ Russia         NA rl_Nasia       Europe    RU
##  2 Acorus c~ NT          not_t~ Russia         NA rl_Nasia       Europe    RU
##  3 Acorus c~ NT          not_t~ Russia         NA rl_Nasia       Europe    RU
##  4 Acorus c~ CR          threa~ Russia         NA rl_Nasia       Europe    RU
##  5 Actaea s~ NT          not_t~ Russia         NA rl_Nasia       Europe    RU
##  6 Actaea s~ LC          not_t~ Russia         NA rl_Nasia       Europe    RU
##  7 Actaea s~ VU          threa~ Russia         NA rl_Nasia       Europe    RU
##  8 Actaea s~ CR          threa~ Russia         NA rl_Nasia       Europe    RU
##  9 Actaea s~ EX          extin~ Russia         NA rl_Nasia       Europe    RU
## 10 Actaea s~ NT          not_t~ Russia         NA rl_Nasia       Europe    RU
## # i 799 more rows
```

Keep threat info for the latest publication year. In Russia, there is no info about publication year.

Keep Russia separated:

```
redlist_phenobs_Russia <- redlist_phenobs %>%
  filter(iso2 == "RU")
```

Remove Russia:

```
redlist_phenobs_noRussia <- redlist_phenobs %>%
  filter(iso2 != "RU")
```

For other countries apart from Russia: If there are several rows for same publication year, pick the most severe red list category.

```
severity_order <- c("CR", "EN", "VU", "NT", "LC")

redlist_phenobs_noRussia_latest <- redlist_phenobs_noRussia %>%
  group_by(name_res, iso2) %>%
  filter(if (all(is.na(Publ_year))) TRUE else
    Publ_year == max(Publ_year, na.rm = TRUE)) %>%
  slice(which.min(match(Redlist_cat, severity_order))) %>%
  ungroup()

redlist_phenobs_noRussia_latest
```

```
## # A tibble: 1,102 x 8
##    name_res  Redlist_cat Threat Country Publ_year redlist_source Continent iso2
##    <chr>     <chr>       <chr>  <chr>       <dbl> <chr>          <chr>     <chr>
##  1 Achillea~ LC          not_t~ Belgium      2019 rl_europe      Europe    BE
##  2 Achillea~ NT          not_t~ Switze~      2016 rl_NationalRL  Europe    CH
##  3 Achillea~ LC          not_t~ Germany      2018 rl_europe      Europe    DE
##  4 Achillea~ LC          not_t~ Estonia      2008 rl_NationalRL  Europe    EE
##  5 Achillea~ LC          not_t~ France       2018 rl_europe      Europe    FR
##  6 Achillea~ LC          not_t~ United~      2014 rl_NationalRL  Europe    GB
##  7 Achillea~ LC          not_t~ Ireland      2016 rl_NationalRL  Europe    IE
##  8 Achillea~ LC          not_t~ Luxemb~      2005 rl_europe      Europe    LU
##  9 Achillea~ LC          not_t~ Nether~      2012 rl_NationalRL  Europe    NL
## 10 Aconitum~ EN          threa~ Austria      1999 rl_NationalRL  Europe    AT
## # i 1,092 more rows
```

Check that there is only one entry for each species and country:

```
redlist_phenobs_noRussia_latest %>%
  group_by(name_res, iso2) %>%
  count() %>% filter(n > 1)
```

```
## # A tibble: 0 x 3
## # Groups:   name_res, iso2 [0]
## # i 3 variables: name_res <chr>, iso2 <chr>, n <int>
```

For Russia:

```
severity_threatened <- c("CR", "EN", "VU")
severity_not_threatened <- c("NT", "LC")

redlist_phenobs_Russia <- redlist_phenobs_Russia %>%
  group_by(name_res) %>%
  # Apply custom logic to pick ONE row but keep all columns
  slice({
    threatened_count <- sum(Threat == "threatened")
    not_threatened_count <- sum(Threat == "not_threatened")

    # If there are more threatened rows than not threatened
```

```
    if (threatened_count > not_threatened_count) {
      # pick the first threatened row
      which.max(Threat == "threatened")
      # Else if there are more not threatened rows
    } else if (not_threatened_count > threatened_count) {
      # pick the first not threatened row
      which.max(Threat == "not_threatened")
    } else {
      # If there is a tie between threatened and not threatened rows,
      # break by severity groups.
      # Count severity categories
      threatened_severity_count <- sum(Redlist_cat %in% severity_threatened)
      not_threatened_severity_count <- sum(Redlist_cat %in% severity_not_threatened)
      # If threatened severity count  not threatened severity count
      if (threatened_severity_count >= not_threatened_severity_count) {
        # Picks the first row with CR/EN/VU
        which.max(Redlist_cat %in% severity_threatened)
        # Else
      } else {
        # Picks the first row with NT/LC
        which.max(Redlist_cat %in% severity_not_threatened)
      }
    }
  }) %>%
  ungroup()
```

For Russia, check that there is only one entry for each species:

```
redlist_phenobs_Russia %>% group_by(name_res) %>%
  count() %>% filter(n > 1)
```

```
## # A tibble: 0 x 2
## # Groups:   name_res [0]
## # i 2 variables: name_res <chr>, n <int>
```

Join noRussia and Russia to get final RL for PhenObs:

```
redlist_phenobs_final <- bind_rows(redlist_phenobs_noRussia_latest,
                                   redlist_phenobs_Russia)
# Note that species that are not threatened in any country
# are absent from redlist_phenobs_final
```

Data prep RL for Germany

```
redlist_Germany <- redlist_phenobs %>%
  filter(iso2 == "DE") %>%
  # Keep threat info for the latest publication year for each sp
  group_by(name_res) %>%
  filter(if (all(is.na(Publ_year))) TRUE else
    Publ_year == max(Publ_year, na.rm = TRUE)) %>%
```

```
  slice(which.min(match(Redlist_cat, severity_order))) %>%
  ungroup()
```

```
redlist_Germany %>% count(Threat, Redlist_cat)
```

```
## # A tibble: 3 x 3
##   Threat         Redlist_cat     n
##   <chr>          <chr>       <int>
## 1 not_threatened LC             81
## 2 threatened     EN             25
## 3 threatened     VU              1
```

Does not make sense to look at RL categories, cause there are 25 EN + 1 VU = 26 threatened, and 81 LC = 81 not threatened.

Join occurrence data and Red List data

Europe

```
redlist_phenobs_final_occ <-
  # Here, all 177 species are included
  occ_matrix_pa %>%
  select(-n_countries) %>%
  pivot_longer(cols = c(2:43), names_to = "iso2", values_to = "occurrence") %>%
  # Here, only the species included in red lists in ANY country (i.e. 154 spp) are included
  left_join(redlist_phenobs_final, by = c("species" = "name_res",
                                          "iso2" = "iso2")) %>%
  # Remove unneded columns
  select(-Redlist_cat, -Country, -Publ_year, -redlist_source, -Continent) %>%
  # Rename
  rename(country = iso2, threat = Threat) %>%
  # Order columns
  select(species, country, occurrence, threat) %>%
  # Set those where threat is NA or not_evaluated_dd (only 2 spp in Russia)
  # as not_threatened
  mutate(threat = case_when(
    threat == "not_evaluated_dd" ~ "not_threatened",
    is.na(threat) ~ "not_threatened",
    TRUE ~ threat
  ))
```

Germany

```
redlist_Germany_occ <-
  # Here, all 177 species are included
  occ_matrix_pa %>%
  select(-n_countries) %>%
```

```
  pivot_longer(cols = c(2:43), names_to = "iso2", values_to = "occurrence") %>%
  filter(iso2 == "DE") %>%
  # Here, only the species included in RL in Germany are included
  left_join(redlist_Germany, by = c("species" = "name_res",
                                    "iso2" = "iso2")) %>%
  # Remove unneded columns
  select(-Redlist_cat, -Country, -Publ_year, -redlist_source, - Continent) %>%
  # Rename
  rename(country = iso2, threat = Threat) %>%
  # Order columns
  select(species, country, occurrence, threat) %>%
  # Set those where Threat is NA or not_evaluated_dd (only 2 spp in Russia)
  # as not_threatened
  mutate(threat = case_when(
    threat == "not_evaluated_dd" ~ "not_threatened",
    is.na(threat) ~ "not_threatened",
    TRUE ~ threat
  ))
```

```
RL_threat_Germany <- redlist_Germany_occ %>%
  filter(occurrence > 0) %>%
  select(species, threat)
```

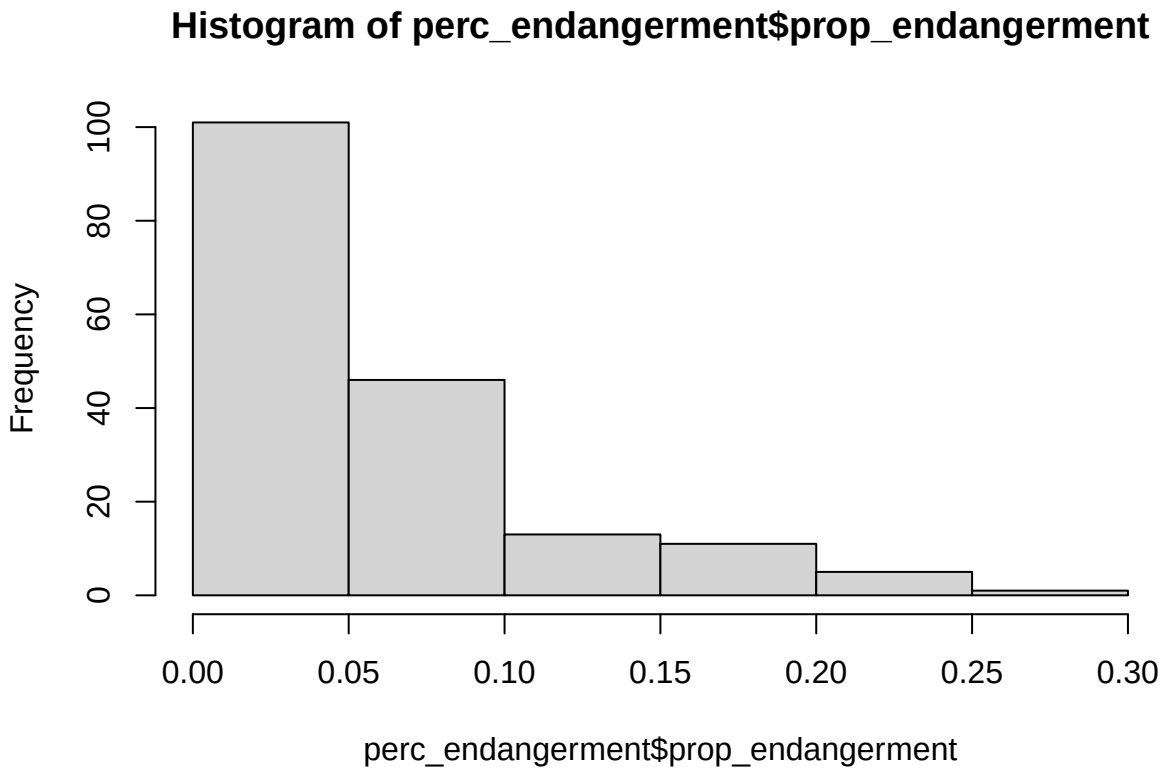
Calculate % endangerment

```
perc_endangerment <- redlist_phenobs_final_occ %>%
  filter(occurrence > 0) %>%  # only countries where species is present
  group_by(species) %>%
  summarise(
    total_countries = n_distinct(country),
    threatened_countries = sum(threat == "threatened"),
    perc_endangerment = (threatened_countries / total_countries) * 100,
    .groups = "drop"
  ) %>%
  arrange(species) %>%
  # Have it also as a proportion
  mutate(prop_endangerment = perc_endangerment / 100)
perc_endangerment
```

```
## # A tibble: 177 x 5
##    species                total_countries threatened_countries perc_endangerment
##    <chr>                            <int>                <int>             <dbl>
##  1 Achillea millefolium                40                    0              0
##  2 Aconitum lycoctonum                 36                    3              8.33
##  3 Acorus calamus                      33                    2              6.06
##  4 Actaea spicata                      38                    1              2.63
##  5 Adonis vernalis                     31                    7             22.6
##  6 Aegopodium podagraria               40                    0              0
##  7 Agrimonia eupatoria                 40                    1              2.5
##  8 Ajuga pyramidalis                   38                   11             28.9
```

```
##  9 Alchemilla vulgaris                36                    0              0
## 10 Alchemilla xanthochlo~             36                    2              5.56
## # i 167 more rows
## # i 1 more variable: prop_endangerment <dbl>
```

```
hist(perc_endangerment$prop_endangerment)
```



Data prep sensitivity data

Flowering

```
sensitivity_data_fl_resolved <- sensitivity_data_fl_resolved %>%
  # Rename columns
  rename(mean_slope_fl = mean_slope, ci_lower_fl = ci_lower,
         ci_upper_fl = ci_upper, slope_var_fl = slope_var) %>%
  # Add variable for "sens" vs "not-sens"
  mutate(sens_01_fl = factor(ifelse(ci_lower_fl > 0 | ci_upper_fl < 0, 1, 0))) %>%
  # Calculate absolute value of mean slope
  mutate(mean_slope_abs_fl = abs(mean_slope_fl))
head(sensitivity_data_fl_resolved)
```

```
##                 Species mean_slope_fl ci_lower_fl ci_upper_fl slope_var_fl
```

```
## 1  Achillea millefolium     -3.693691 -11.5164009   9.5933164    26.168179
## 2   Aconitum lycoctonum      8.375899   1.0970659  16.8947249    14.630941
## 3        Acorus calamus     -8.439830 -22.0938994   2.3558606    43.236584
## 4        Actaea spicata      3.851829  -0.4832343   6.6896977     3.294084
## 5       Adonis vernalis     -1.975730  -8.2505266   5.7754348    11.564267
## 6 Aegopodium podagraria     -2.720275  -6.7086444   0.1762956     2.608240
##   n_gardens temp_min temp_max temp_mean n_data             name_res sens_01_fl
## 1        13 5.443750 14.60667 11.263438     36  Achillea millefolium          0
## 2         6 8.662500 12.90000 11.039112     23   Aconitum lycoctonum          1
## 3         6 9.180000 11.95625 10.398144     22        Acorus calamus          0
## 4         4 8.556250 12.61250 10.398509     19        Actaea spicata          0
## 5         6 8.106667 11.06667  9.735369     26       Adonis vernalis          0
## 6         7 4.293333 11.95625 10.288601     28 Aegopodium podagraria          0
##   mean_slope_abs_fl
## 1          3.693691
## 2          8.375899
## 3          8.439830
## 4          3.851829
## 5          1.975730
## 6          2.720275
```

GSL

```
sensitivity_data_GSL_resolved <- sensitivity_data_GSL_resolved %>%
  # Rename columns
  rename(mean_slope_GSL = mean_slope, ci_lower_GSL = ci_lower,
         ci_upper_GSL = ci_upper, slope_var_GSL= slope_var) %>%
  # Add variable for "sens" vs "not-sens"
  mutate(sens_01_GSL = factor(ifelse(ci_lower_GSL > 0 | ci_upper_GSL < 0, 1, 0))) %>%
  # Calculate absolute value of mean slope
  mutate(mean_slope_abs_GSL = abs(mean_slope_GSL))
head(sensitivity_data_GSL_resolved)
```

```
##                 Species mean_slope_GSL ci_lower_GSL ci_upper_GSL slope_var_GSL
## 1  Achillea millefolium     -12.387450  -21.6108791      2.11973      40.58041
## 2   Aconitum lycoctonum      11.343354   -0.6043651     20.33298      26.74367
## 3        Acorus calamus       3.771969  -22.3456312     21.79345      92.43691
## 4        Actaea spicata      -7.493619  -24.8443976     11.04222      86.26684
## 5       Adonis vernalis       4.278485   -9.0467881     14.96125      40.17993
## 6 Aegopodium podagraria      15.129065   -5.4147556     33.46060      77.69152
##   n_gardens temp_min temp_max temp_mean n_data             name_res
## 1        11 8.641667 15.78333  11.55208     28  Achillea millefolium
## 2         5 8.641667 12.43333  10.90625     20   Aconitum lycoctonum
## 3         8 4.833333 12.17500  10.70417     28        Acorus calamus
## 4         4 8.641667 12.43333  11.01548     21        Actaea spicata
## 5         6 8.641667 12.79167  11.09938     27       Adonis vernalis
## 6         8 4.858333 12.17500  10.77742     31 Aegopodium podagraria
##   sens_01_GSL mean_slope_abs_GSL
## 1           0          12.387450
## 2           0          11.343354
## 3           0           3.771969
## 4           0           7.493619
```

```
## 5           0           4.278485
## 6           0          15.129065
```

Prepare datasets for models

Europe

Sensitivity + % endangerment

```
sens_per_endanger <- perc_endangerment %>%
  left_join(
    sensitivity_data_fl_resolved %>%
      select(name_res, mean_slope_fl, sens_01_fl, mean_slope_abs_fl, ci_lower_fl,
             ci_upper_fl, slope_var_fl),
    by = c("species" = "name_res"),
    ) %>%
  left_join(
    sensitivity_data_GSL_resolved %>%
      select(name_res, mean_slope_GSL, sens_01_GSL, mean_slope_abs_GSL,
             ci_lower_GSL, ci_upper_GSL, slope_var_GSL),
    by = c("species" = "name_res")
    )
sens_per_endanger %>% filter(is.na(perc_endangerment))  %>% nrow()
```

```
## [1] 0
```

```
sens_per_endanger %>% filter(is.na(mean_slope_fl)) %>% nrow() # 6 spp
```

```
## [1] 6
```

```
sens_per_endanger %>% filter(is.na(mean_slope_GSL)) %>% nrow() # 28 spp
```

```
## [1] 28
```

Sensitivity + % endangerment + stress

```
sens_per_endanger_stress <- sens_per_endanger %>%
  left_join(stress_resolved, by = c("species" = "name_res")) %>%
  select(-Species)
```

Sensitivity + % endangerment + Ellenberg tendency of change

Deal with non-numeric values in indicator values

Columns OEK_L - OEK_S and HAEUF_AE_change

```
ellenberg <- ellenberg %>%
  mutate(across(OEK_L:OEK_S, ~ parse_number(.)))
```

```
## Warning: There were 7 warnings in `mutate()`.
## The first warning was:
## i In argument: `across(OEK_L:OEK_S, ~parse_number(.))`.
## Caused by warning:
## ! 28 parsing failures.
## row col expected actual
## 184  -- a number      x
## 200  -- a number      x
## 676  -- a number      x
## 744  -- a number      x
## 798  -- a number      x
## ... ... ........ ......
## See problems(...) for more details.
## i Run `dplyr::last_dplyr_warnings()` to see the 6 remaining warnings.
```

Join with sens_per_endanger:

```
sens_per_endanger_stress_ell <- sens_per_endanger_stress %>%
  left_join(ellenberg_resolved %>%
              select(name_res, HAEUF_AE_change) %>%
              rename(change = HAEUF_AE_change) %>%
              group_by(name_res) %>%
              # For duplicates, get the mean
              summarise(change = as.integer(mean(change, na.rm = TRUE)),
                        .groups = "drop"),
            by = c("species" = "name_res"))
```

Germany: sensitivity + threat

```
sens_threat_Germany <- RL_threat_Germany %>%
  mutate(threat = ifelse(threat == "threatened", 1, 0)) %>%
  left_join(
    sensitivity_data_fl_resolved %>%
      select(name_res, mean_slope_fl, sens_01_fl, mean_slope_abs_fl, ci_lower_fl,
             ci_upper_fl, slope_var_fl),
    by = c("species" = "name_res")
    ) %>%
  left_join(
    sensitivity_data_GSL_resolved %>%
      select(name_res, mean_slope_GSL, sens_01_GSL, mean_slope_abs_GSL,
             ci_lower_GSL, ci_upper_GSL, slope_var_GSL) ,
    by = c("species" = "name_res")
    )
sens_threat_Germany %>% filter(is.na(threat))
```

```
## # A tibble: 0 x 14
## # i 14 variables: species <chr>, threat <dbl>, mean_slope_fl <dbl>,
```

```
## #   sens_01_fl <fct>, mean_slope_abs_fl <dbl>, ci_lower_fl <dbl>,
## #   ci_upper_fl <dbl>, slope_var_fl <dbl>, mean_slope_GSL <dbl>,
## #   sens_01_GSL <fct>, mean_slope_abs_GSL <dbl>, ci_lower_GSL <dbl>,
## #   ci_upper_GSL <dbl>, slope_var_GSL <dbl>
```

```
sens_threat_Germany %>% filter(is.na(mean_slope_fl)) # 6 spp
```

```
## # A tibble: 6 x 14
##   species          threat mean_slope_fl sens_01_fl mean_slope_abs_fl ci_lower_fl
##   <chr>             <dbl>         <dbl> <fct>                  <dbl>       <dbl>
## 1 Agrimonia eupat~      0            NA <NA>                      NA          NA
## 2 Helosciadium re~      0            NA <NA>                      NA          NA
## 3 Knautia arvensis      0            NA <NA>                      NA          NA
## 4 Oenothera bienn~      0            NA <NA>                      NA          NA
## 5 Saxifraga rotun~      0            NA <NA>                      NA          NA
## 6 Thymus praecox        0            NA <NA>                      NA          NA
## # i 8 more variables: ci_upper_fl <dbl>, slope_var_fl <dbl>,
## #   mean_slope_GSL <dbl>, sens_01_GSL <fct>, mean_slope_abs_GSL <dbl>,
## #   ci_lower_GSL <dbl>, ci_upper_GSL <dbl>, slope_var_GSL <dbl>
```

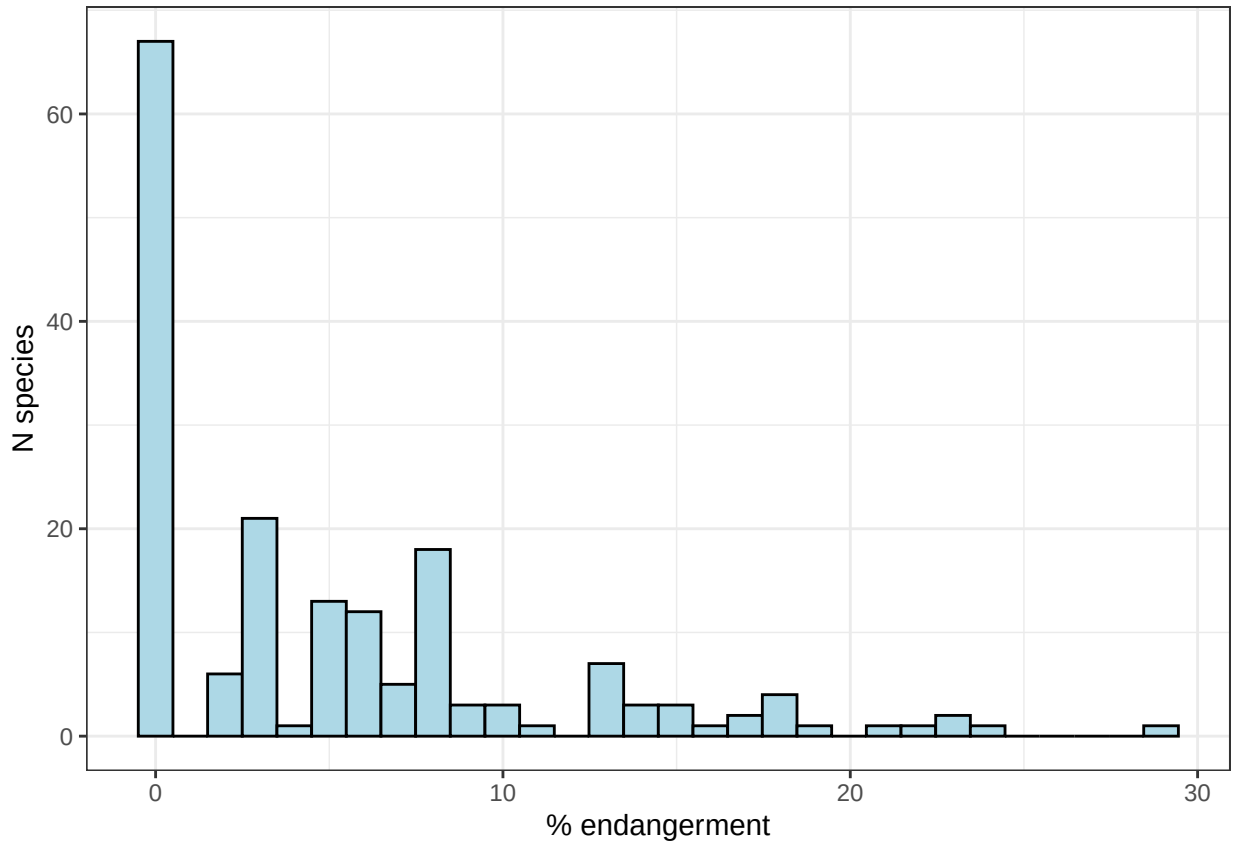
```
sens_threat_Germany %>% filter(is.na(mean_slope_GSL)) # 27 spp
```

```
## # A tibble: 27 x 14
##    species         threat mean_slope_fl sens_01_fl mean_slope_abs_fl ci_lower_fl
##    <chr>            <dbl>         <dbl> <fct>                  <dbl>       <dbl>
##  1 Agrimonia eupa~      0        NA     <NA>                  NA          NA
##  2 Ajuga pyramida~      0        -0.696 0                      0.696      -4.53
##  3 Alchemilla vul~      0         6.67  0                      6.67       -0.907
##  4 Antennaria dio~      1        -7.96  1                      7.96      -10.6
##  5 Anthericum ram~      1         8.43  0                      8.43       -6.23
##  6 Bergenia purpu~      0        -0.971 0                      0.971      -8.05
##  7 Calluna vulgar~      0        -0.210 0                      0.210      -2.42
##  8 Daphne laureola      0        -6.92  1                      6.92       -8.39
##  9 Fuchsia magell~      0         6.02  1                      6.02        0.498
## 10 Galium mollugo       0        -7.42  1                      7.42       -9.46
## # i 17 more rows
## # i 8 more variables: ci_upper_fl <dbl>, slope_var_fl <dbl>,
## #   mean_slope_GSL <dbl>, sens_01_GSL <fct>, mean_slope_abs_GSL <dbl>,
## #   ci_lower_GSL <dbl>, ci_upper_GSL <dbl>, slope_var_GSL <dbl>
```

Distributions

```
ggplot(sens_per_endanger, aes(x = perc_endangerment)) +
  geom_histogram(color = "black", fill = "lightblue") +
  theme_bw() +
  xlab("% endangerment") + ylab("N species")
```

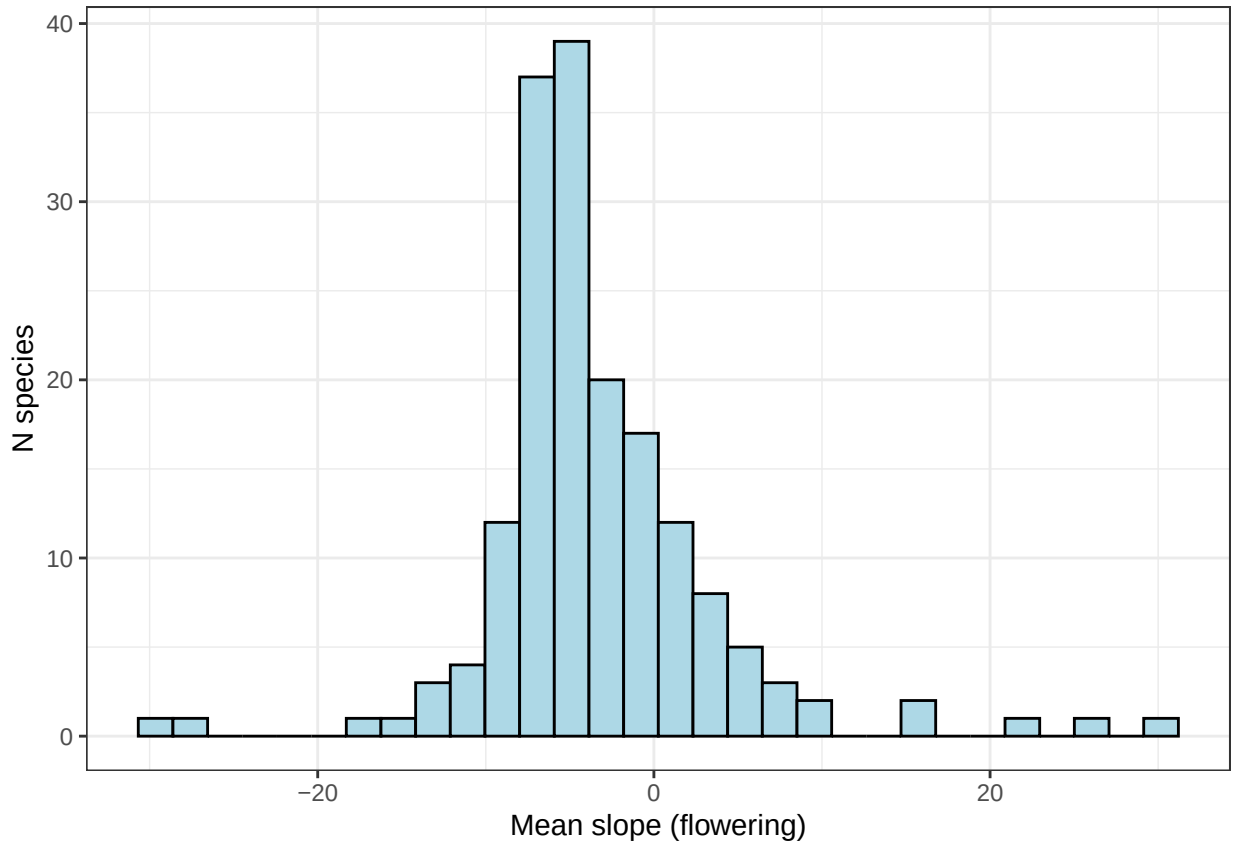
```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```




```
ggplot(sens_per_endanger, aes(x = mean_slope_fl)) +
  geom_histogram(color = "black", fill = "lightblue") +
  theme_bw() +
  xlab("Mean slope (flowering)") + ylab("N species")
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

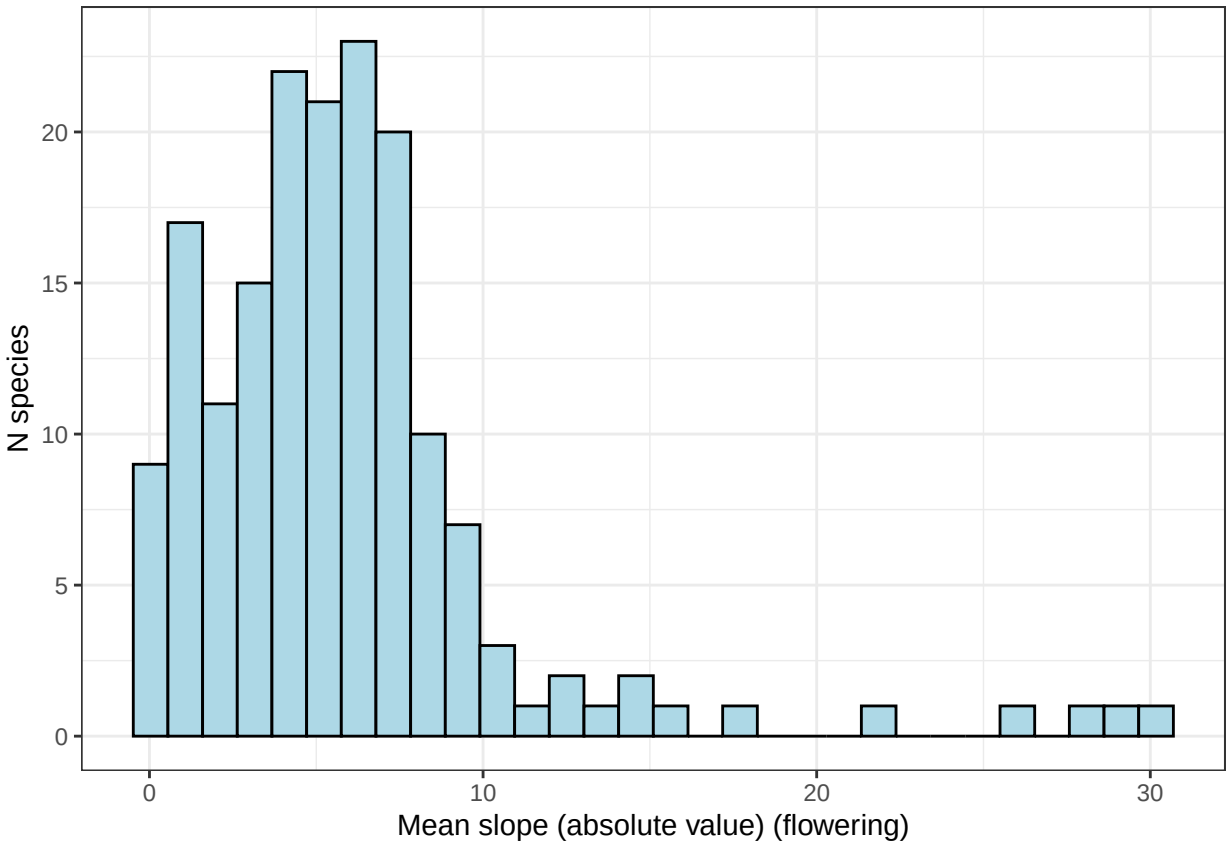
```
## Warning: Removed 6 rows containing non-finite outside the scale range
## (`stat_bin()`).
```




```
ggplot(sens_per_endanger, aes(x = mean_slope_abs_fl)) +
  geom_histogram(color = "black", fill = "lightblue") +
  theme_bw() +
  xlab("Mean slope (absolute value) (flowering)") + ylab("N species")
```

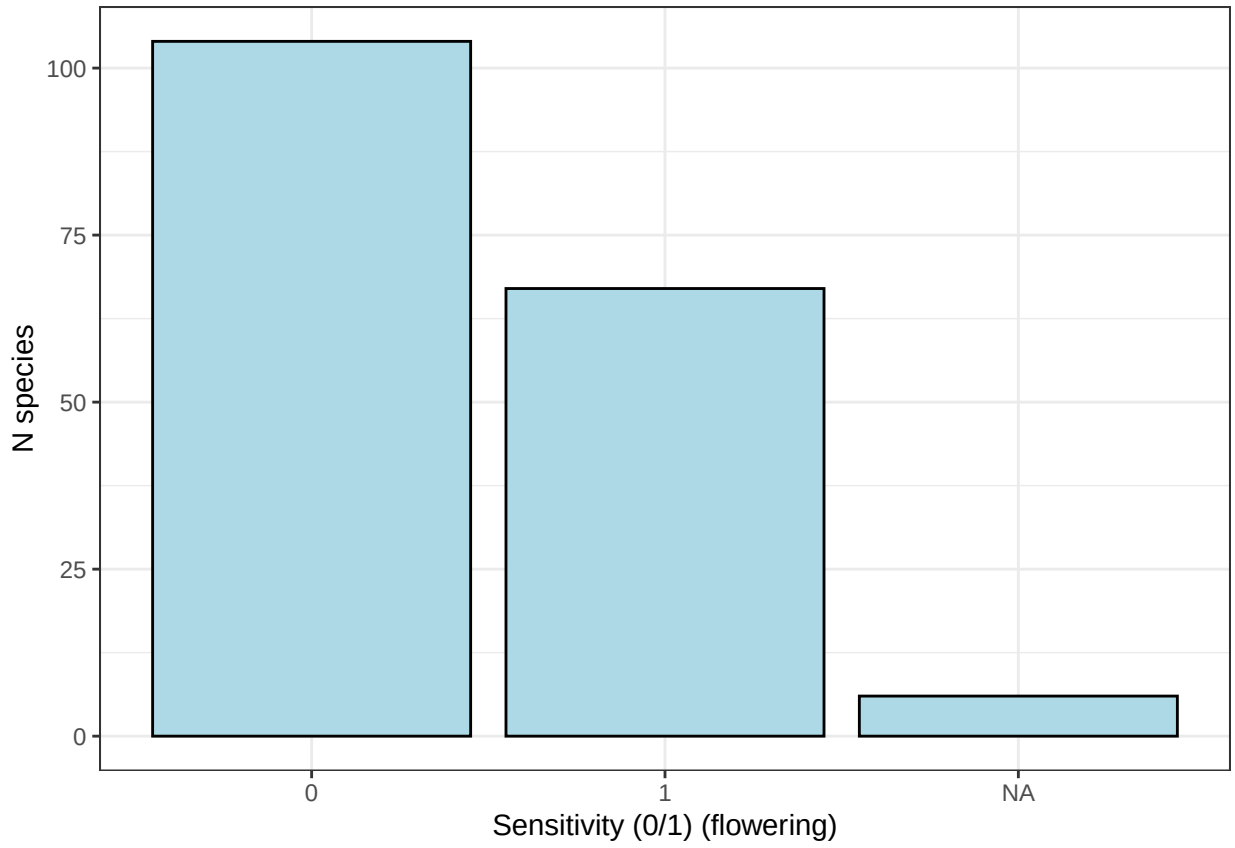
```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

```
## Warning: Removed 6 rows containing non-finite outside the scale range
## (`stat_bin()`).
```




```
ggplot(sens_per_endanger, aes(x = sens_01_fl)) +
  geom_histogram(color = "black", fill = "lightblue", stat = "count") +
  theme_bw() +
  xlab("Sensitivity (0/1) (flowering)") + ylab("N species")
```

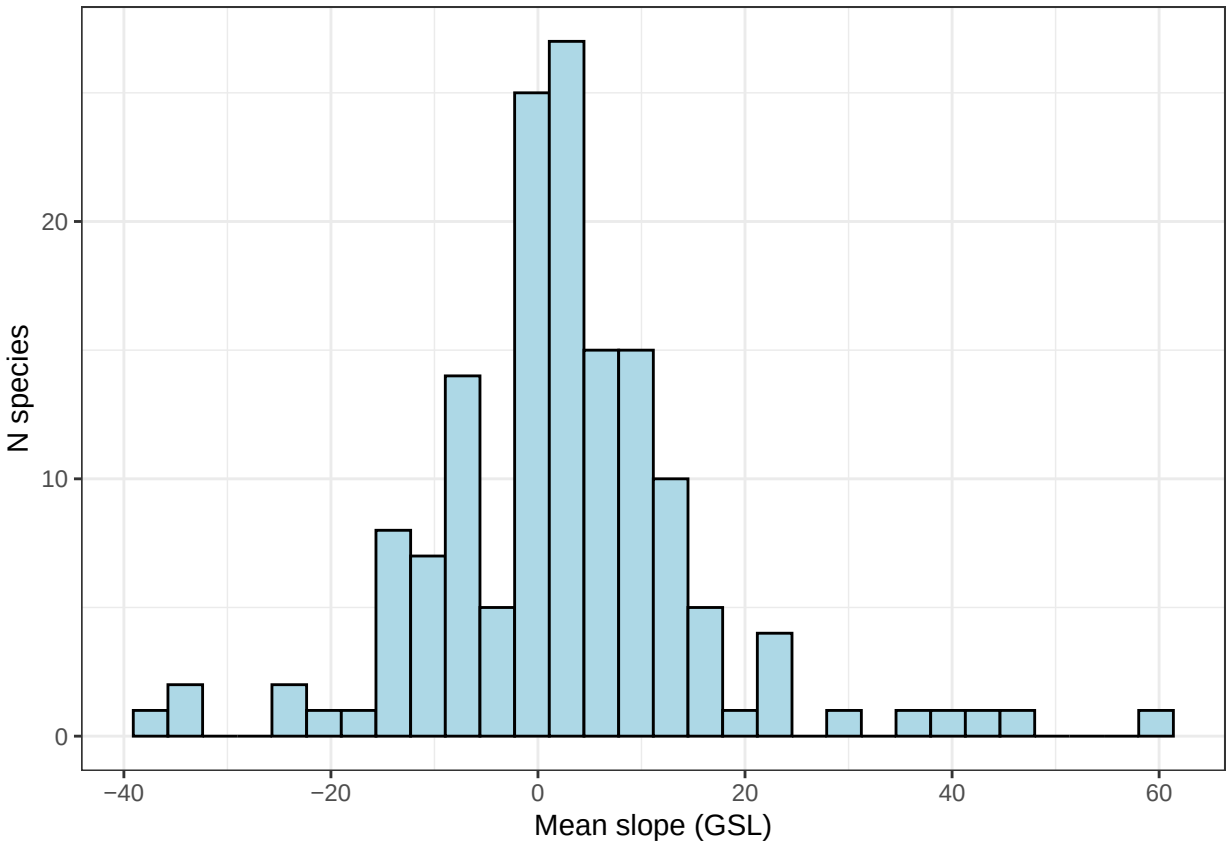
```
## Warning in geom_histogram(color = "black", fill = "lightblue", stat = "count"):
## Ignoring unknown parameters: `binwidth` and `bins`
```




```
ggplot(sens_per_endanger, aes(x = mean_slope_GSL)) +
  geom_histogram(color = "black", fill = "lightblue") +
  theme_bw() +
  xlab("Mean slope (GSL)") + ylab("N species")
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

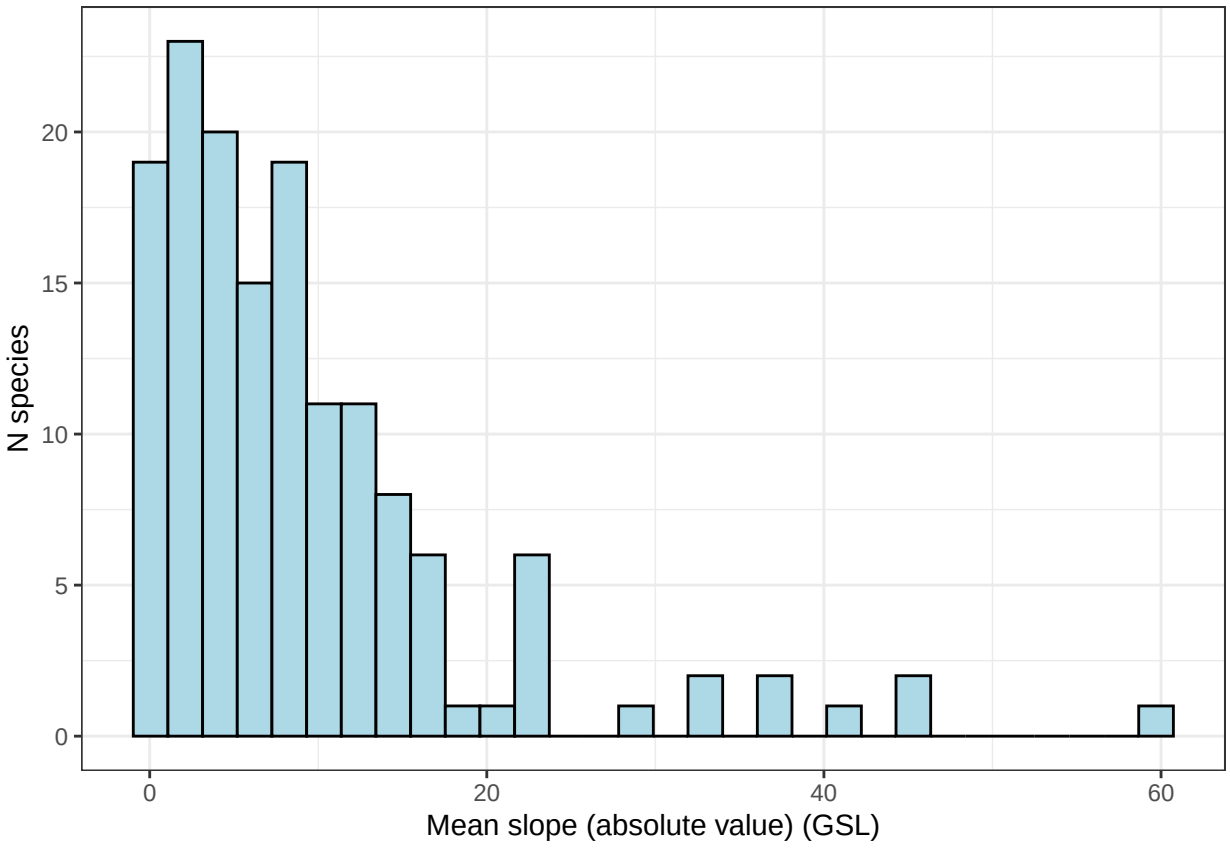
```
## Warning: Removed 28 rows containing non-finite outside the scale range
## (`stat_bin()`).
```




```
ggplot(sens_per_endanger, aes(x = mean_slope_abs_GSL)) +
  geom_histogram(color = "black", fill = "lightblue") +
  theme_bw() +
  xlab("Mean slope (absolute value) (GSL)") + ylab("N species")
```

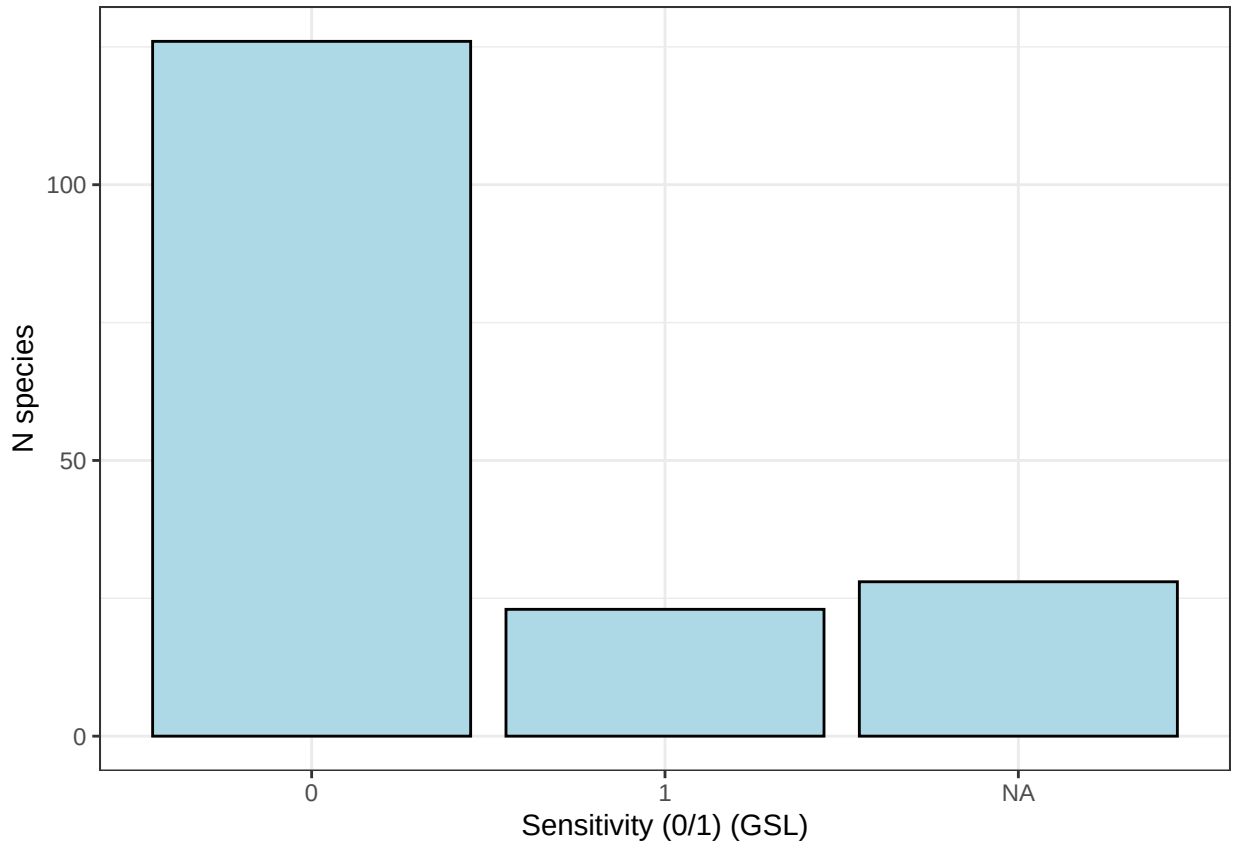
```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```

```
## Warning: Removed 28 rows containing non-finite outside the scale range
## (`stat_bin()`).
```

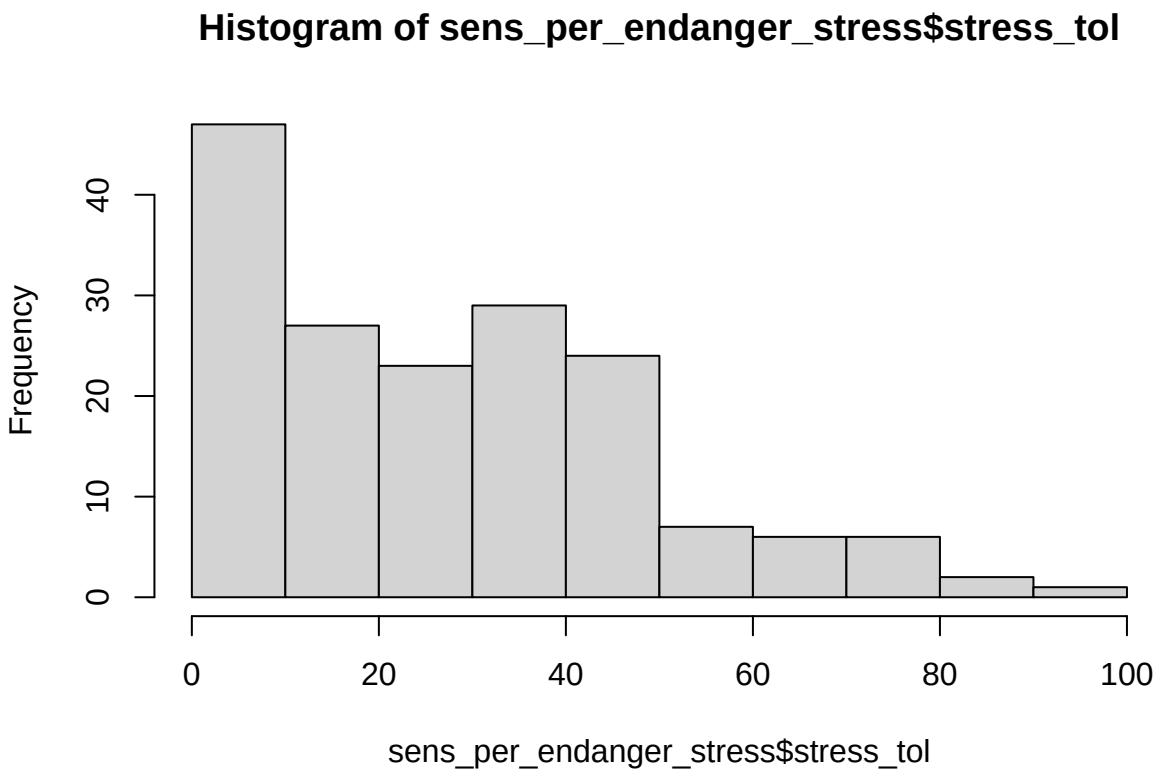



```
ggplot(sens_per_endanger, aes(x = sens_01_GSL)) +
  geom_histogram(color = "black", fill = "lightblue", stat = "count") +
  theme_bw() +
  xlab("Sensitivity (0/1) (GSL)") + ylab("N species")
```

```
## Warning in geom_histogram(color = "black", fill = "lightblue", stat = "count"):
## Ignoring unknown parameters: `binwidth` and `bins`
```




```
hist(sens_per_endanger_stress$stress_tol)
```



Models endangerment ~ sensitivity

Europe

Filtering slope_var < 50 - clearer patterns.

Using only zero-inflated models.

Added prediction plots only for models with any significant effects.

Sensitive or not

```
model_threat_sens_01_fl_50 <- glmmTMB(
  prop_endangerment ~ sens_01_fl, family = "beta_family", ziformula=~.,
  data = sens_per_endanger %>%  filter(slope_var_fl < 50))
model_threat_sens_01_GSL_50 <- glmmTMB(
  prop_endangerment ~ sens_01_GSL, family = "beta_family", ziformula=~.,
  data = sens_per_endanger %>% filter(slope_var_GSL < 50))
summary(model_threat_sens_01_fl_50)
```

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ sens_01_fl
## Zero inflation:                     ~.
```

```
## Data: sens_per_endanger %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    -119.2    -103.7      64.6    -129.2       158
##
##
## Dispersion parameter for beta family (): 28.1
##
## Conditional model:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -2.3331     0.0800 -29.164   <2e-16 ***
## sens_01_fl1  -0.2222     0.1288  -1.724   0.0846 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.5596     0.2089  -2.679  0.00739 **
## sens_01_fl1   0.1149     0.3306   0.348  0.72810
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_threat_sens_01_GSL_50)
```

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ sens_01_GSL
## Zero inflation:                     ~.
## Data: sens_per_endanger %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     -29.7     -19.5      19.9     -39.7        52
##
##
## Dispersion parameter for beta family (): 38.8
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -2.3781     0.1104 -21.537   <2e-16 ***
## sens_01_GSL1  -0.4920     0.2801  -1.757   0.0789 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -0.3137     0.3018  -1.039    0.299
## sens_01_GSL1   0.3137     0.6515   0.482    0.630
```

Mean slopes

Flowering (absolute value)

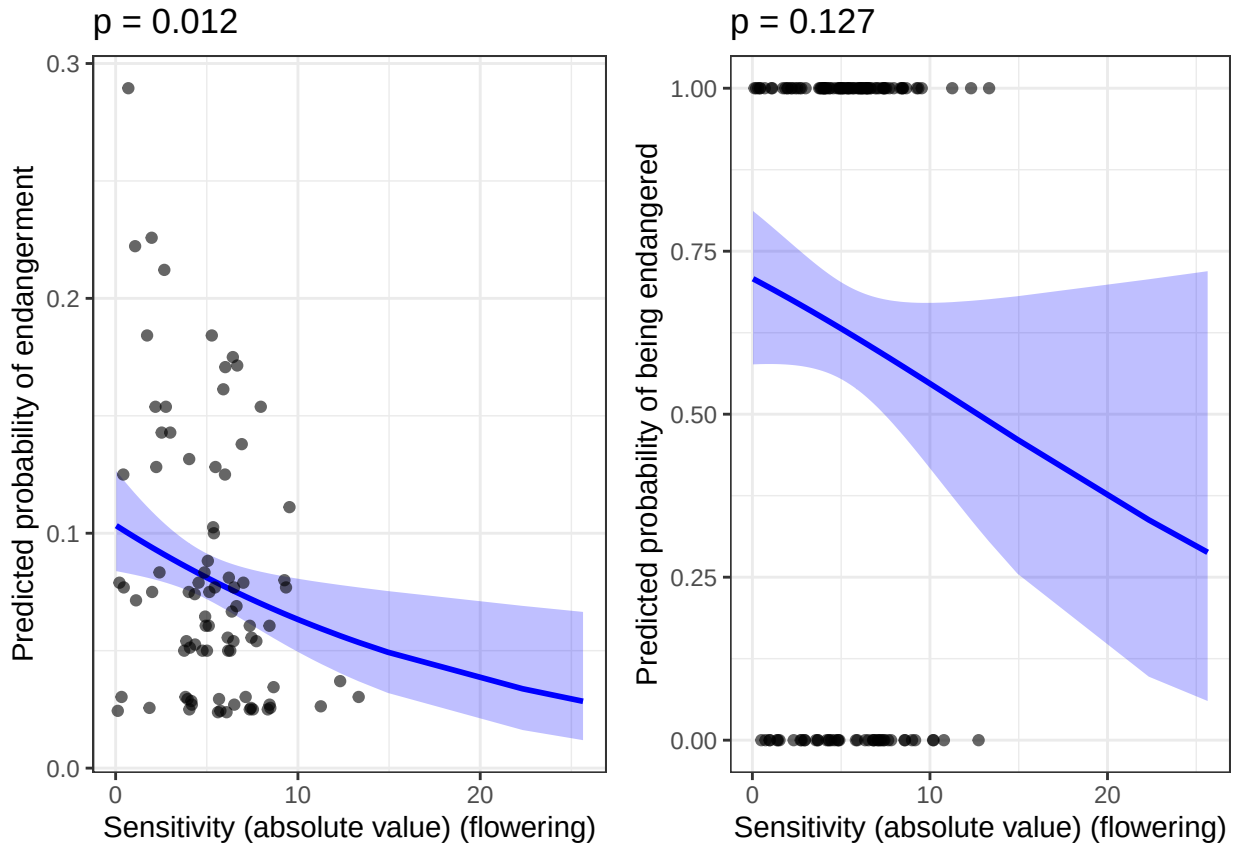
```
model_threat_mean_slope_all_fl_zi_50 <- glmmTMB(
  prop_endangerment ~ mean_slope_abs_fl, family = "beta_family",
  ziformula=~.,
  data = sens_per_endanger %>% filter(slope_var_fl < 50))
summary(model_threat_mean_slope_all_fl_zi_50)
```

All slopes

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ mean_slope_abs_fl
## Zero inflation:                     ~.
## Data: sens_per_endanger %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    -124.7    -109.2      67.4    -134.7       158
##
##
## Dispersion parameter for beta family ():   29
##
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -2.15996    0.11755 -18.375   <2e-16 ***
## mean_slope_abs_fl -0.05345    0.02130  -2.509   0.0121 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -0.88702    0.29555  -3.001  0.00269 **
## mean_slope_abs_fl  0.06987    0.04578   1.526  0.12701
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
grid.arrange(
  ggplot() +
    geom_ribbon(data = data.frame(ggpredict(model_threat_mean_slope_all_fl_zi_50,
                                            type = "fixed",
                                            terms="mean_slope_abs_fl [all]")),
                aes(x = x, y = predicted, ymin = conf.low, ymax = conf.high),
                alpha = 0.25, fill = "blue") +
    geom_line(data = data.frame(ggpredict(model_threat_mean_slope_all_fl_zi_50,
                                          type = "fixed",
                                          terms="mean_slope_abs_fl [all]")),
              aes(x = x, y = predicted), color = "blue", linewidth = 1) +
    geom_point(data = sens_per_endanger %>%
                 filter(mean_slope_fl < 0 & slope_var_fl < 50 &
                          prop_endangerment > 0),
               aes(x = mean_slope_abs_fl, y = prop_endangerment), alpha = 0.6) +
    theme_bw() + ggtitle("p = 0.012") +
    xlab("Sensitivity (absolute value) (flowering)") +
    ylab("Predicted probability of endangerment"),
```

```
  ggplot() +
    geom_ribbon(data = data.frame(ggpredict(model_threat_mean_slope_all_fl_zi_50,
                                            type = "zi_prob",
                                            terms="mean_slope_abs_fl [all]")),
                aes(x = x, y = 1-predicted, ymin = 1-conf.low,
                    ymax = 1-conf.high),
                alpha = 0.25, fill = "blue") +
    geom_line(data = data.frame(ggpredict(model_threat_mean_slope_all_fl_zi_50,
                                          type = "zi_prob",
                                          terms="mean_slope_abs_fl [all]")),
              aes(x = x, y = 1-predicted), color = "blue", linewidth = 1) +
    geom_point(data = sens_per_endanger %>%
                 filter(mean_slope_fl < 0 & slope_var_fl < 50) %>%
                 mutate(endangerment_01 = ifelse(prop_endangerment == 0, 0, 1)),
               aes(x = mean_slope_abs_fl, y = endangerment_01), alpha = 0.6) +
    theme_bw() + ggtitle("p = 0.127") +
    xlab("Sensitivity (absolute value) (flowering)") +
    ylab("Predicted probability of being endangered"),
  ncol = 2)
```




```
model_threat_mean_slope_neg_fl_zi_50 <- glmmTMB(
  prop_endangerment ~ mean_slope_abs_fl, family = "beta_family",
```

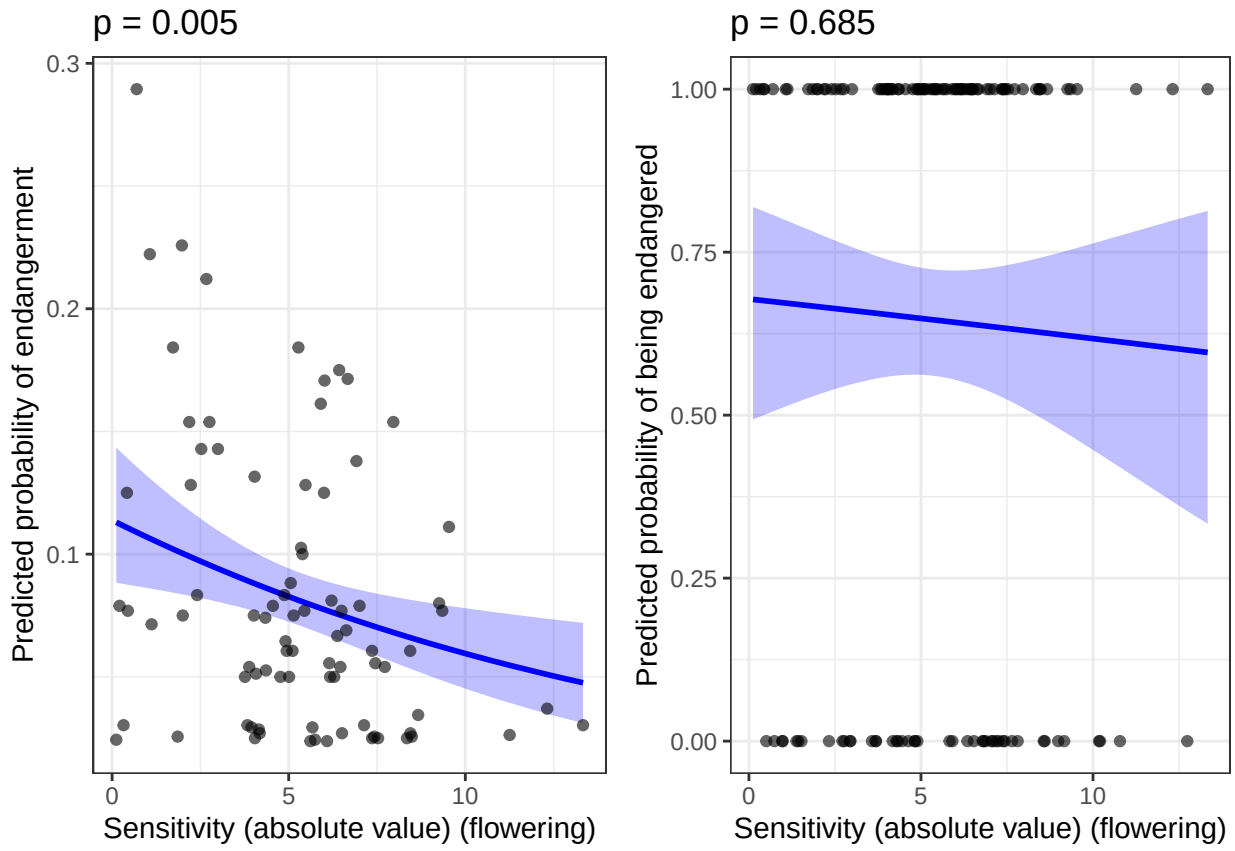
```
  ziformula=~.,
  data = sens_per_endanger %>% filter(mean_slope_fl < 0 & slope_var_fl < 50))
summary(model_threat_mean_slope_neg_fl_zi_50)
```

Negative slopes only

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ mean_slope_abs_fl
## Zero inflation:                     ~.
## Data: sens_per_endanger %>% filter(mean_slope_fl < 0 & slope_var_fl <      50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    -107.2     -92.9      58.6    -117.2       125
##
##
## Dispersion parameter for beta family (): 28.5
##
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -2.05239    0.14208 -14.445  < 2e-16 ***
## mean_slope_abs_fl -0.07079    0.02506  -2.825  0.00472 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -0.74546    0.39934  -1.867   0.0619 .
## mean_slope_abs_fl  0.02668    0.06568   0.406   0.6846
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
grid.arrange(
  ggplot() +
    geom_ribbon(data = data.frame(ggpredict(model_threat_mean_slope_neg_fl_zi_50,
                                            type = "fixed",
                                            terms="mean_slope_abs_fl [all]")),
                aes(x = x, y = predicted, ymin = conf.low, ymax = conf.high),
                alpha = 0.25, fill = "blue") +
    geom_line(data = data.frame(ggpredict(model_threat_mean_slope_neg_fl_zi_50,
                                          type = "fixed",
                                          terms="mean_slope_abs_fl [all]")),
              aes(x = x, y = predicted), color = "blue", linewidth = 1) +
    geom_point(data = sens_per_endanger %>%
                 filter(mean_slope_fl < 0 & slope_var_fl < 50 &
                          prop_endangerment > 0),
               aes(x = mean_slope_abs_fl, y = prop_endangerment), alpha = 0.6) +
    theme_bw() + ggtitle("p = 0.005") +
    xlab("Sensitivity (absolute value) (flowering)") +
    ylab("Predicted probability of endangerment"),
  ggplot() +
    geom_ribbon(data = data.frame(ggpredict(model_threat_mean_slope_neg_fl_zi_50,
                                            type = "zi_prob",
                                            terms="mean_slope_abs_fl [all]")),
```

```
                aes(x = x, y = 1-predicted, ymin = 1-conf.low,
                    ymax = 1-conf.high),
                alpha = 0.25, fill = "blue") +
    geom_line(data = data.frame(ggpredict(model_threat_mean_slope_neg_fl_zi_50,
                                          type = "zi_prob",
                                          terms="mean_slope_abs_fl [all]")),
              aes(x = x, y = 1-predicted), color = "blue", linewidth = 1) +
    geom_point(data = sens_per_endanger %>%
                 filter(mean_slope_fl < 0 & slope_var_fl < 50) %>%
                 mutate(endangerment_01 = ifelse(prop_endangerment == 0, 0, 1)),
               aes(x = mean_slope_abs_fl, y = endangerment_01), alpha = 0.6) +
    theme_bw() + ggtitle("p = 0.685") +
    xlab("Sensitivity (absolute value) (flowering)") +
    ylab("Predicted probability of being endangered"),
  ncol = 2)
```



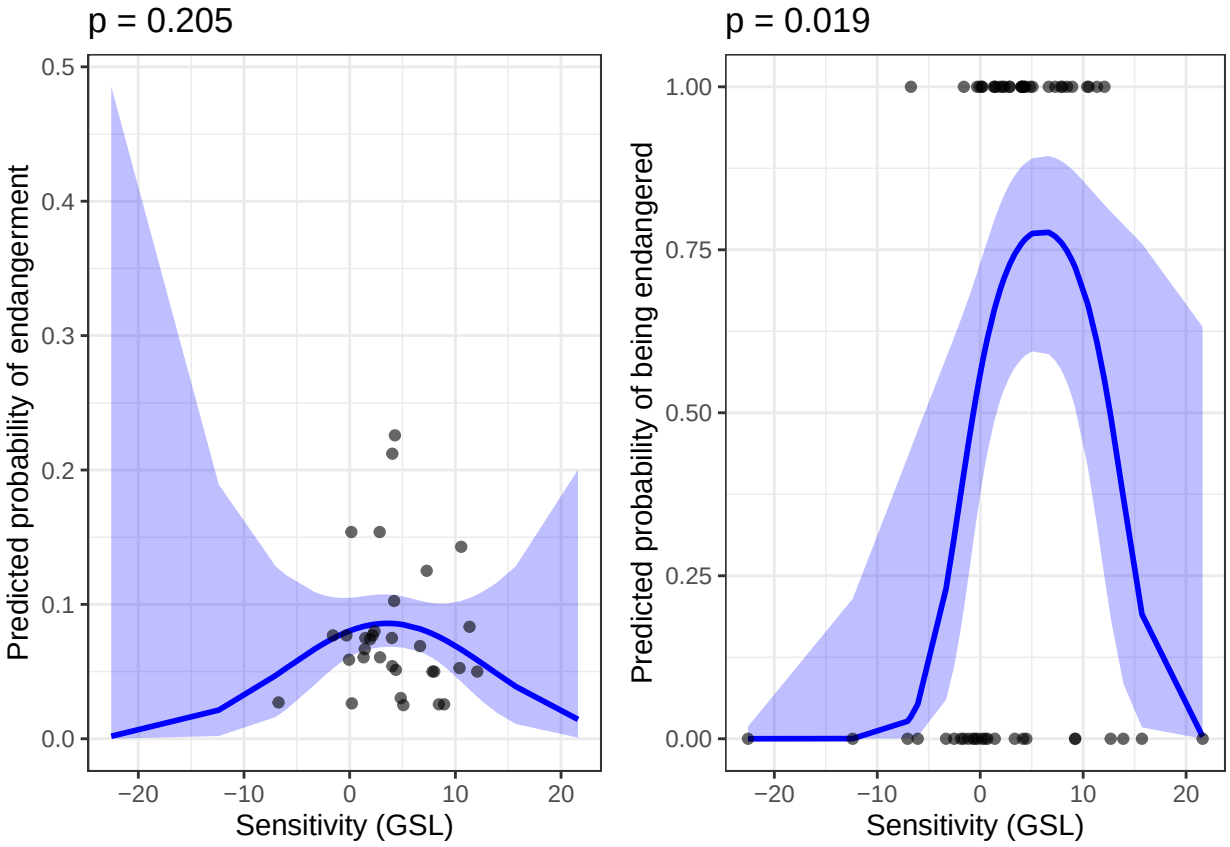

```
model_threat_mean_slope_GSL_zi_50 <- glmmTMB(
  prop_endangerment ~ mean_slope_GSL + I(mean_slope_GSL^2),
  family = "beta_family", ziformula=~.,
  data = sens_per_endanger %>% filter(slope_var_GSL < 50))
summary(model_threat_mean_slope_GSL_zi_50)
```

GSL (quadratic model)

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ mean_slope_GSL + I(mean_slope_GSL^2)
## Zero inflation:                     ~.
## Data: sens_per_endanger %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     -38.4     -24.1      26.2     -52.4        50
##
##
## Dispersion parameter for beta family (): 36.7
##
## Conditional model:
##                      Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -2.438689   0.150631  -16.19   <2e-16 ***
## mean_slope_GSL       0.041076   0.043258    0.95    0.342
## I(mean_slope_GSL^2) -0.005728   0.004547   -1.26    0.208
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -0.24040    0.38424  -0.626   0.5316
## mean_slope_GSL      -0.34124    0.14108  -2.419   0.0156 *
## I(mean_slope_GSL^2)  0.02854    0.01213   2.353   0.0186 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
grid.arrange(
  ggplot() +
    geom_ribbon(data = data.frame(ggpredict(model_threat_mean_slope_GSL_zi_50,
                                            type = "fixed",
                                            terms="mean_slope_GSL [all]")),
                aes(x = x, y = predicted, ymin = conf.low, ymax = conf.high),
                alpha = 0.25, fill = "blue") +
    geom_line(data = data.frame(ggpredict(model_threat_mean_slope_GSL_zi_50,
                                          type = "fixed",
                                          terms="mean_slope_GSL [all]")),
              aes(x = x, y = predicted), color = "blue", linewidth = 1) +
    geom_point(data = sens_per_endanger %>%
                 filter(slope_var_GSL < 50 &
                          prop_endangerment > 0),
               aes(x = mean_slope_GSL, y = prop_endangerment), alpha = 0.6) +
    theme_bw() + ggtitle("p = 0.205") +
    xlab("Sensitivity (GSL)") +
    ylab("Predicted probability of endangerment"),
  ggplot() +
    geom_ribbon(data = data.frame(ggpredict(model_threat_mean_slope_GSL_zi_50,
                                            type = "zi_prob",
                                            terms="mean_slope_GSL [all]")),
                aes(x = x, y = 1-predicted, ymin = 1-conf.low,
                    ymax = 1-conf.high),
                alpha = 0.25, fill = "blue") +
    geom_line(data = data.frame(ggpredict(model_threat_mean_slope_GSL_zi_50,
                                          type = "zi_prob",
```

```
                                        terms="mean_slope_GSL [all]")),
            aes(x = x, y = 1-predicted), color = "blue", linewidth = 1) +
  geom_point(data = sens_per_endanger %>%
               filter(slope_var_GSL < 50) %>%
               mutate(endangerment_01 = ifelse(prop_endangerment == 0, 0, 1)),
             aes(x = mean_slope_GSL, y = endangerment_01), alpha = 0.6) +
  theme_bw() + ggtitle("p = 0.019") +
  xlab("Sensitivity (GSL)") +
  ylab("Predicted probability of being endangered"),
ncol = 2)
```



Germany (all NS)

Filtering slope_var < 50 - clearer patterns.

Variable "threat" is 0/1, cannot use zero-inflated models, just binomial models.

Sensitive or not

```
model_threat_sens_01_fl_50_G <- glmmTMB(
  threat ~ sens_01_fl, family = binomial,
  data = sens_threat_Germany %>% filter(slope_var_fl < 50))
model_threat_sens_01_GSL_50_G <- glmmTMB(
```

```
  threat ~ sens_01_GSL, family = binomial,
  data = sens_threat_Germany %>% filter(slope_var_GSL < 50))
summary(model_threat_sens_01_fl_50_G)
```

```
##  Family: binomial  ( logit )
## Formula:          threat ~ sens_01_fl
## Data: sens_threat_Germany %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     139.2     145.3     -67.6     135.2       158
##
##
## Conditional model:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -1.7918     0.2887  -6.207 5.41e-10 ***
## sens_01_fl1   0.1431     0.4501   0.318    0.751
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_threat_sens_01_GSL_50_G)
```

```
##  Family: binomial  ( logit )
## Formula:          threat ~ sens_01_GSL
## Data: sens_threat_Germany %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##      58.9      62.9     -27.5      54.9        53
##
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.3291     0.3749  -3.546 0.000392 ***
## sens_01_GSL1  -0.2803     0.8605  -0.326 0.744629
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Mean slopes

Flowering (absolute value)

```
model_threat_mean_slope_all_fl_50_G <- glmmTMB(
  threat ~ mean_slope_abs_fl,
  family = binomial,
  data = sens_threat_Germany %>% filter(slope_var_fl < 50))
summary(model_threat_mean_slope_all_fl_50_G)
```

All slopes

```
##  Family: binomial  ( logit )
## Formula:          threat ~ mean_slope_abs_fl
## Data: sens_threat_Germany %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     139.1     145.3     -67.6     135.1       158
##
##
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -1.86556    0.40234  -4.637 3.54e-06 ***
## mean_slope_abs_fl  0.02519    0.06339   0.397    0.691
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
model_threat_mean_slope_neg_fl_50_G <- glmmTMB(
  threat ~ mean_slope_abs_fl,
  family = binomial,
  data = sens_threat_Germany %>% filter(mean_slope_fl < 0 & slope_var_fl < 50))
summary(model_threat_mean_slope_neg_fl_50_G)
```

Negative slopes only

```
##  Family: binomial  ( logit )
## Formula:          threat ~ mean_slope_abs_fl
## Data: sens_threat_Germany %>% filter(mean_slope_fl < 0 & slope_var_fl <
##     50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     114.9     120.6     -55.5     110.9       126
##
##
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -1.74709    0.53318  -3.277  0.00105 **
## mean_slope_abs_fl  0.01144    0.08889   0.129  0.89763
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
model_threat_mean_slope_GSL_50_G <- glmmTMB(
  threat ~ mean_slope_GSL + I(mean_slope_GSL^2), family = binomial,
  data = sens_threat_Germany %>% filter(slope_var_GSL < 50))
summary(model_threat_mean_slope_GSL_50_G)
```

GSL (quadratic model)

```
##  Family: binomial  ( logit )
```

```
## Formula:          threat ~ mean_slope_GSL + I(mean_slope_GSL^2)
## Data: sens_threat_Germany %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##      57.0      63.0     -25.5      51.0        52
##
##
## Conditional model:
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -1.43543    0.49861  -2.879  0.00399 **
## mean_slope_GSL       0.23753    0.18284   1.299  0.19389
## I(mean_slope_GSL^2) -0.02204    0.01667  -1.322  0.18615
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Correlation mean slope flowering - GSL

```
cor(sens_per_endanger$mean_slope_fl, sens_per_endanger$mean_slope_GSL,
    use = "complete.obs")
```

```
## [1] -0.09893824
```

```
cor(sens_per_endanger$mean_slope_abs_fl, sens_per_endanger$mean_slope_abs_GSL,
    use = "complete.obs")
```

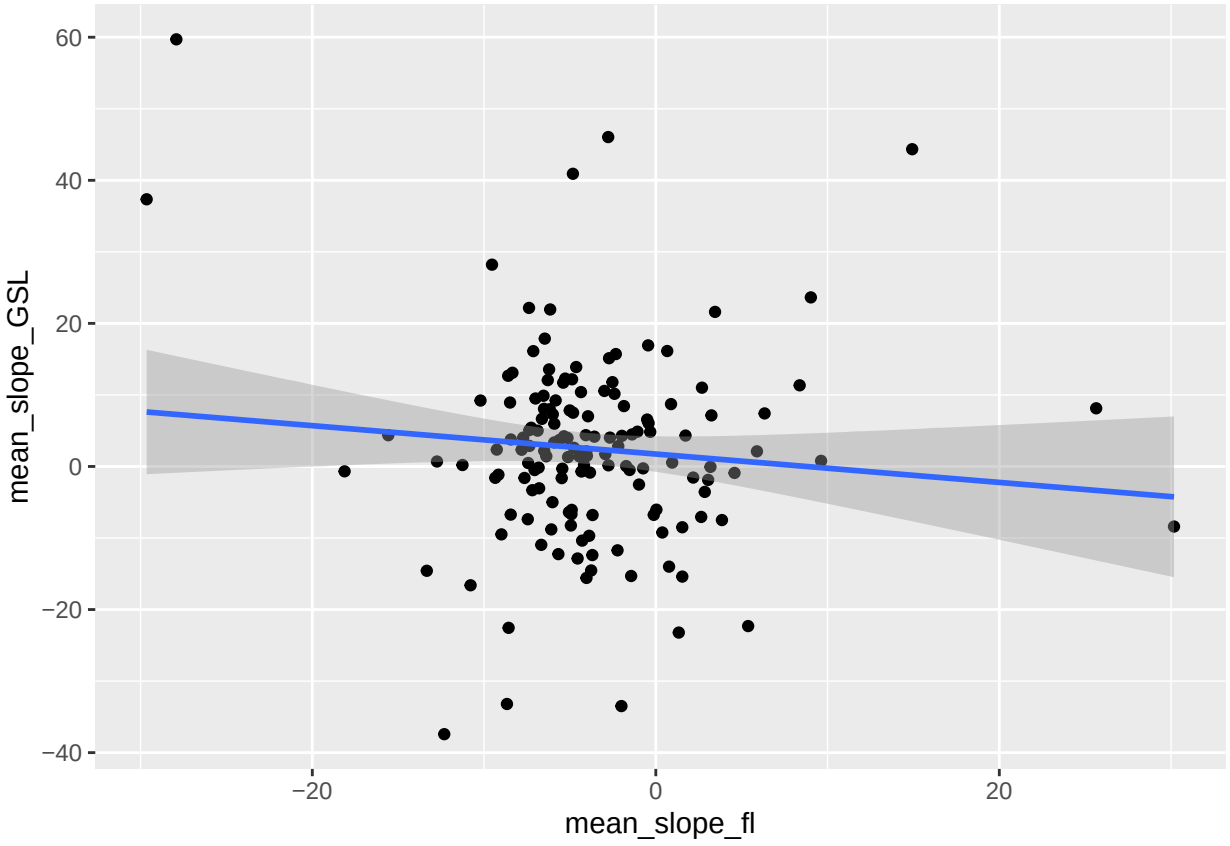
```
## [1] 0.3099428
```

```
ggplot(sens_per_endanger, aes(x = mean_slope_fl, y = mean_slope_GSL)) +
  geom_point() + geom_smooth(method = "lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 28 rows containing non-finite outside the scale range
## (`stat_smooth()`).

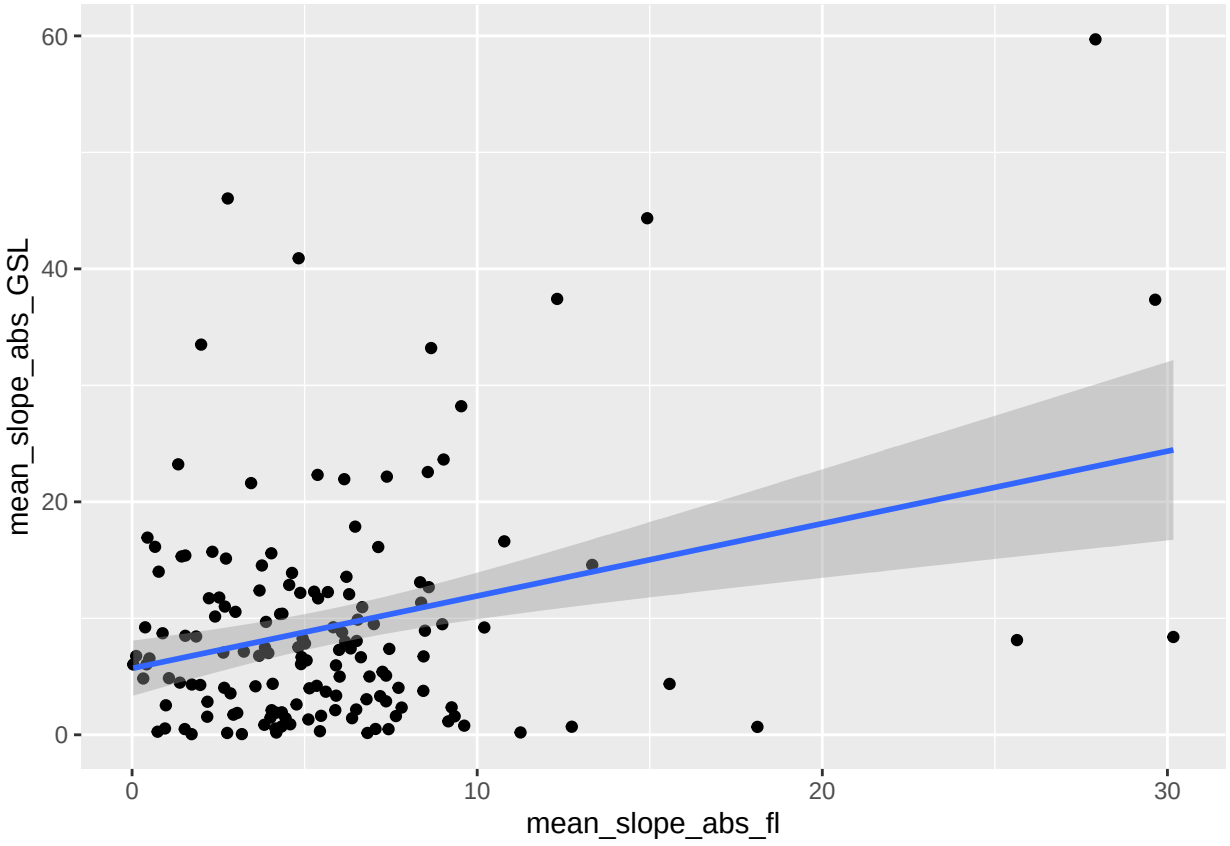
## Warning: Removed 28 rows containing missing values or values outside the scale range
## (`geom_point()`).
```




```
ggplot(sens_per_endanger, aes(x = mean_slope_abs_fl, y = mean_slope_abs_GSL)) +
  geom_point() + geom_smooth(method = "lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 28 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 28 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Models stress tolerance

Endangerment ~ stress tolerance

```
model_threat_stress <- glmmTMB(prop_endangerment ~ stress_tol,
                               family = "beta_family", ziformula=~.,
                               data = sens_per_endanger_stress)
summary(model_threat_stress)
```

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ stress_tol
## Zero inflation:                     ~.
## Data: sens_per_endanger_stress
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    -123.9    -108.2      67.0    -133.9       167
##
##
## Dispersion parameter for beta family (): 25.9
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
```

```
## (Intercept) -2.418100   0.104507 -23.138   <2e-16 ***
## stress_tol   0.001042   0.002757   0.378    0.706
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.296304   0.253753  -1.168    0.243
## stress_tol  -0.009445   0.007600  -1.243    0.214
```

Endangerment ~ stress tolerance * sensitivity

Filtering slope_var < 50 - clearer patterns.

Sensitive or not

```
model_threat_sens_01_fl_50_stress <- glmmTMB(
  prop_endangerment ~ stress_tol * sens_01_fl, family = "beta_family",
  ziformula=~., data = sens_per_endanger_stress %>% filter(slope_var_fl < 50))
model_threat_sens_01_GSL_50_stress <- glmmTMB(
  prop_endangerment ~ stress_tol * sens_01_GSL, family = "beta_family",
  ziformula=~., data = sens_per_endanger_stress %>% filter(slope_var_GSL < 50))
summary(model_threat_sens_01_fl_50_stress)
```

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ stress_tol * sens_01_fl
## Zero inflation:                     ~.
## Data: sens_per_endanger_stress %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    -118.7     -90.9      68.3    -136.7       152
##
##
## Dispersion parameter for beta family (): 28.7
##
## Conditional model:
##                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)            -2.3229232  0.1302292 -17.837   <2e-16 ***
## stress_tol             -0.0004239  0.0032365  -0.131   0.8958
## sens_01_fl1            -0.4206970  0.2165924  -1.942   0.0521 .
## stress_tol:sens_01_fl1  0.0076962  0.0060481   1.272   0.2032
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)            -0.223685   0.368240  -0.607    0.544
## stress_tol             -0.012395   0.010543  -1.176    0.240
## sens_01_fl1            -0.141658   0.532365  -0.266    0.790
## stress_tol:sens_01_fl1  0.007441   0.015857   0.469    0.639
```

```r
summary(model_threat_sens_01_GSL_50_stress)
```

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ stress_tol * sens_01_GSL
## Zero inflation:                     ~.
## Data: sens_per_endanger_stress %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     -26.9      -8.5      22.4     -44.9        48
##
##
## Dispersion parameter for beta family (): 45.5
##
## Conditional model:
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)             -2.493730   0.137285 -18.165   <2e-16 ***
## stress_tol               0.005436   0.004646   1.170   0.2420
## sens_01_GSL1            -0.979851   0.463642  -2.113   0.0346 *
## stress_tol:sens_01_GSL1  0.026090   0.016124   1.618   0.1056
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)             -0.442433   0.435461  -1.016    0.310
## stress_tol               0.006804   0.016436   0.414    0.679
## sens_01_GSL1             0.206567   0.944164   0.219    0.827
## stress_tol:sens_01_GSL1  0.004591   0.033652   0.136    0.891
```

Mean slopes

Flowering (absolute)

```r
# Model without ZI
model_threat_mean_slope_all_fl_50_stress <- glmmTMB(
  cbind(threatened_countries, total_countries - threatened_countries) ~
    stress_tol * mean_slope_abs_fl, family = binomial,
  data = sens_per_endanger_stress %>% filter(slope_var_fl < 50))
# Model with ZI
model_threat_mean_slope_all_fl_zi_50_stress <- glmmTMB(
  prop_endangerment ~ stress_tol * mean_slope_abs_fl, family = "beta_family",
  ziformula=~.,
  data = sens_per_endanger_stress %>% filter(slope_var_fl < 50))
summary(model_threat_mean_slope_all_fl_50_stress)
```

All slopes

```
##  Family: binomial  ( logit )
## Formula:
```

```
## cbind(threatened_countries, total_countries - threatened_countries) ~
##     stress_tol * mean_slope_abs_fl
## Data: sens_per_endanger_stress %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     642.5     654.8    -317.2     634.5       157
##
##
## Conditional model:
##                                Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -2.3312271  0.2108752 -11.055  < 2e-16 ***
## stress_tol                   -0.0055417  0.0050436  -1.099 0.271872
## mean_slope_abs_fl            -0.1419244  0.0394215  -3.600 0.000318 ***
## stress_tol:mean_slope_abs_fl  0.0023514  0.0009788   2.402 0.016289 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_threat_mean_slope_all_fl_zi_50_stress)
```

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ stress_tol * mean_slope_abs_fl
## Zero inflation:                     ~.
## Data: sens_per_endanger_stress %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    -128.1    -100.4      73.1    -146.1       152
##
##
## Dispersion parameter for beta family (): 31.1
##
## Conditional model:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -1.804958   0.213231  -8.465  < 2e-16 ***
## stress_tol                   -0.010406   0.005092  -2.043 0.041004 *
## mean_slope_abs_fl            -0.138003   0.040069  -3.444 0.000573 ***
## stress_tol:mean_slope_abs_fl  0.002775   0.001072   2.588 0.009644 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -0.492377   0.560848  -0.878    0.380
## stress_tol                   -0.012595   0.015164  -0.831    0.406
## mean_slope_abs_fl             0.029041   0.088434   0.328    0.743
## stress_tol:mean_slope_abs_fl  0.001008   0.002620   0.385    0.701
```

```
# Model without ZI
model_threat_mean_slope_neg_fl_50_stress <- glmmTMB(
  cbind(threatened_countries, total_countries - threatened_countries) ~
    stress_tol * mean_slope_abs_fl, family = binomial,
  data = sens_per_endanger_stress %>% filter(mean_slope_fl < 0 & slope_var_fl < 50))
```

```r
# Model with ZI
model_threat_mean_slope_neg_fl_zi_50_stress <- glmmTMB(
  prop_endangerment ~ stress_tol * mean_slope_abs_fl, family = "beta_family",
  ziformula=~.,
  data = sens_per_endanger_stress %>% filter(mean_slope_fl < 0 & slope_var_fl < 50))
summary(model_threat_mean_slope_neg_fl_50_stress)
```

Negative slopes only

```
##  Family: binomial  ( logit )
## Formula:
## cbind(threatened_countries, total_countries - threatened_countries) ~
##     stress_tol * mean_slope_abs_fl
## Data: sens_per_endanger_stress %>% filter(mean_slope_fl < 0 & slope_var_fl <
##     50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     521.3     532.7    -256.6     513.3       124
##
##
## Conditional model:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                 -2.329859   0.250609  -9.297  < 2e-16 ***
## stress_tol                  -0.002060   0.005716  -0.360  0.71861
## mean_slope_abs_fl           -0.137495   0.044985  -3.056  0.00224 **
## stress_tol:mean_slope_abs_fl  0.001853   0.001078   1.719  0.08561 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
summary(model_threat_mean_slope_neg_fl_zi_50_stress)
```

```
##  Family: beta  ( logit )
## Formula:          prop_endangerment ~ stress_tol * mean_slope_abs_fl
## Zero inflation:                     ~.
## Data: sens_per_endanger_stress %>% filter(mean_slope_fl < 0 & slope_var_fl <
##     50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##    -110.0     -84.4      64.0    -128.0       119
##
##
## Dispersion parameter for beta family (): 29.9
##
## Conditional model:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                 -1.793427   0.272143  -6.590  4.4e-11 ***
## stress_tol                  -0.007306   0.005952  -1.227  0.21965
## mean_slope_abs_fl           -0.137522   0.048154  -2.856  0.00429 **
## stress_tol:mean_slope_abs_fl  0.002166   0.001188   1.823  0.06825 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
```

```
##                                Estimate Std. Error z value Pr(>|z|)
## (Intercept)                    0.120528   0.729344   0.165    0.869
## stress_tol                    -0.026233   0.020202  -1.299    0.194
## mean_slope_abs_fl             -0.068484   0.114593  -0.598    0.550
## stress_tol:mean_slope_abs_fl  0.002270   0.003378   0.672    0.502
```

```
# Model without ZI
model_threat_mean_slope_GSL_50_stress <- glmmTMB(
  cbind(threatened_countries, total_countries - threatened_countries) ~
    stress_tol * (mean_slope_GSL + I(mean_slope_GSL^2)), family = binomial,
  data = sens_per_endanger_stress %>% filter(slope_var_GSL < 50))
# Model with ZI
model_threat_mean_slope_GSL_zi_50_stress <- glmmTMB(
  prop_endangerment ~ stress_tol * (mean_slope_GSL + I(mean_slope_GSL^2)),
  family = "beta_family", ziformula=~.,
  data = sens_per_endanger_stress %>% filter(slope_var_GSL < 50))
summary(model_threat_mean_slope_GSL_50_stress)
```

GSL (quadratic model)

```
##  Family: binomial  ( logit )
## Formula:
## cbind(threatened_countries, total_countries - threatened_countries) ~
##     stress_tol * (mean_slope_GSL + I(mean_slope_GSL^2))
## Data: sens_per_endanger_stress %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     183.9     196.1     -85.9     171.9        51
##
##
## Conditional model:
##                                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)                    -3.1339349  0.2715678 -11.540   <2e-16 ***
## stress_tol                      0.0003758  0.0109166   0.034   0.9725
## mean_slope_GSL                  0.2565510  0.1169760   2.193   0.0283 *
## I(mean_slope_GSL^2)            -0.0282730  0.0110194  -2.566   0.0103 *
## stress_tol:mean_slope_GSL      -0.0006565  0.0041869  -0.157   0.8754
## stress_tol:I(mean_slope_GSL^2)  0.0002821  0.0003645   0.774   0.4389
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_threat_mean_slope_GSL_zi_50_stress)
```

```
##  Family: beta  ( logit )
## Formula:
## prop_endangerment ~ stress_tol * (mean_slope_GSL + I(mean_slope_GSL^2))
## Zero inflation:                      ~.
## Data: sens_per_endanger_stress %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
```

```
##     -33.5      -7.0      29.8     -59.5        44
##
##
## Dispersion parameter for beta family (): 45.6
##
## Conditional model:
##                                 Estimate Std. Error z value Pr(>|z|)
## (Intercept)                   -2.5461223  0.2054492 -12.393   <2e-16 ***
## stress_tol                     0.0050706  0.0083786   0.605   0.5451
## mean_slope_GSL                 0.0860784  0.0776925   1.108   0.2679
## I(mean_slope_GSL^2)           -0.0154763  0.0081211  -1.906   0.0567 .
## stress_tol:mean_slope_GSL     -0.0024858  0.0035595  -0.698   0.4850
## stress_tol:I(mean_slope_GSL^2)  0.0005174  0.0003470   1.491   0.1360
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                                 Estimate Std. Error z value Pr(>|z|)
## (Intercept)                   -0.2894637  0.5690131  -0.509    0.611
## stress_tol                     0.0030377  0.0230337   0.132    0.895
## mean_slope_GSL                -0.3006658  0.2261351  -1.330    0.184
## I(mean_slope_GSL^2)            0.0217859  0.0196458   1.109    0.267
## stress_tol:mean_slope_GSL     -0.0017021  0.0090730  -0.188    0.851
## stress_tol:I(mean_slope_GSL^2)  0.0002681  0.0007375   0.364    0.716
```

Models Ellenberg tendency of change ~ sensitivity

Filtering slope_var < 50 - clearer patterns.

Sensitive or not

```
model_ell_sens_01_fl_50 <- glmmTMB(change ~ sens_01_fl,
                                   family = "poisson",
                                   data = sens_per_endanger_stress_ell %>%
                                     filter(slope_var_fl < 50))
model_ell_sens_01_GSL_50 <- glmmTMB(change ~ sens_01_GSL,
                                   family = "poisson",
                                   data = sens_per_endanger_stress_ell %>%
                                     filter(slope_var_GSL < 50))
summary(model_ell_sens_01_fl_50)
```

```
##  Family: poisson  ( log )
## Formula:          change ~ sens_01_fl
## Data: sens_per_endanger_stress_ell %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     422.4     427.9    -209.2     418.4       113
##
##
## Conditional model:
```

```
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.37191    0.06019  22.792   <2e-16 ***
## sens_01_fl1  0.05787    0.09456   0.612    0.541
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_ell_sens_01_GSL_50)
```

```
##  Family: poisson  ( log )
## Formula:          change ~ sens_01_GSL
## Data: sens_per_endanger_stress_ell %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     154.0     157.5     -75.0     150.0        40
##
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   1.39384    0.08671   16.07   <2e-16 ***
## sens_01_GSL1  0.17013    0.17543    0.97    0.332
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Mean slopes

Flowering (absolute)

```
# Model without ZI
model_ell_mean_slope_all_fl_50 <- glmmTMB(change ~ mean_slope_abs_fl,
                                    family = "poisson",
  data = sens_per_endanger_stress_ell %>%
    filter(slope_var_fl < 50))
# Model with ZI
model_ell_mean_slope_all_fl_zi_50 <- glmmTMB(change ~ mean_slope_abs_fl,
                                    family = "poisson",
                                          ziformula=~.,
  data = sens_per_endanger_stress_ell %>%
    filter(slope_var_fl < 50))
summary(model_ell_mean_slope_all_fl_50)
```

All slopes

```
##  Family: poisson  ( log )
## Formula:          change ~ mean_slope_abs_fl
## Data: sens_per_endanger_stress_ell %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     422.7     428.2    -209.3     418.7       113
##
##
```

```
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       1.371902   0.096611  14.200   <2e-16 ***
## mean_slope_abs_fl 0.004616   0.016907   0.273    0.785
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_ell_mean_slope_all_fl_zi_50)
```

```
##  Family: poisson  ( log )
## Formula:          change ~ mean_slope_abs_fl
## Zero inflation:          ~.
## Data: sens_per_endanger_stress_ell %>% filter(slope_var_fl < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     426.7     437.7    -209.3     418.7       111
##
##
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)       1.371902   0.096611  14.200   <2e-16 ***
## mean_slope_abs_fl 0.004616   0.016907   0.273    0.785
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)       -2.302e+01  1.714e+04  -0.001    0.999
## mean_slope_abs_fl  7.197e-02  2.770e+03   0.000    1.000
```

```
# Model without ZI
model_ell__mean_slope_neg_fl_50 <- glmmTMB(change ~ mean_slope_abs_fl,
                                    family = "poisson",
  data = sens_per_endanger_stress_ell %>%
    filter(mean_slope_fl < 0 & slope_var_fl < 50))
# Model with ZI
model_ell__mean_slope_neg_fl_zi_50 <- glmmTMB(change ~ mean_slope_abs_fl,
                                    family = "poisson",
                                          ziformula=~.,
  data = sens_per_endanger_stress_ell %>%
    filter(mean_slope_fl < 0 & slope_var_fl < 50))
summary(model_ell__mean_slope_neg_fl_50)
```

Negative slopes only

```
##  Family: poisson  ( log )
## Formula:          change ~ mean_slope_abs_fl
## Data:
## sens_per_endanger_stress_ell %>% filter(mean_slope_fl < 0 & slope_var_fl <
##     50)
```

```
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     349.2     354.3    -172.6     345.2        92
##
##
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)        1.33273    0.11351  11.741   <2e-16 ***
## mean_slope_abs_fl  0.01167    0.01906   0.613     0.54
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_ell__mean_slope_neg_fl_zi_50)
```

```
##  Family: poisson  ( log )
## Formula:          change ~ mean_slope_abs_fl
## Zero inflation:          ~.
## Data:
## sens_per_endanger_stress_ell %>% filter(mean_slope_fl < 0 & slope_var_fl <
##     50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     353.2     363.4    -172.6     345.2        90
##
##
## Conditional model:
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)        1.33273    0.11351  11.741   <2e-16 ***
## mean_slope_abs_fl  0.01167    0.01906   0.613     0.54
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                    Estimate Std. Error z value Pr(>|z|)
## (Intercept)        -20.8486 11487.4848  -0.002    0.999
## mean_slope_abs_fl   -0.3806  3204.9824   0.000    1.000
```

GSL (quadratic model)

```
# Model without ZI
model_ell_mean_slope_GSL_50 <- glmmTMB(change ~ mean_slope_GSL +
                                         I(mean_slope_GSL^2),
                                       family = "poisson",
                                           data = sens_per_endanger_stress_ell %>%
                                             filter(slope_var_GSL < 50))
# Model with ZI
model_ell_mean_slope_GSL_zi_50 <- glmmTMB(change ~ mean_slope_GSL +
                                            I(mean_slope_GSL^2),
                                       family = "poisson",
                                       ziformula=~.,
                                              data = sens_per_endanger_stress_ell %>%
```

```
                                                filter(slope_var_GSL < 50))
summary(model_ell_mean_slope_GSL_50)
```

```
##  Family: poisson  ( log )
## Formula:          change ~ mean_slope_GSL + I(mean_slope_GSL^2)
## Data: sens_per_endanger_stress_ell %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     155.2     160.4     -74.6     149.2        39
##
##
## Conditional model:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)          1.3885848  0.0909701  15.264   <2e-16 ***
## mean_slope_GSL      -0.0031621  0.0094483  -0.335    0.738
## I(mean_slope_GSL^2)  0.0008110  0.0006037   1.343    0.179
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(model_ell_mean_slope_GSL_zi_50)
```

```
##  Family: poisson  ( log )
## Formula:          change ~ mean_slope_GSL + I(mean_slope_GSL^2)
## Zero inflation:          ~.
## Data: sens_per_endanger_stress_ell %>% filter(slope_var_GSL < 50)
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     161.2     171.6     -74.6     149.2        36
##
##
## Conditional model:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)          1.3885848  0.0909701  15.264   <2e-16 ***
## mean_slope_GSL      -0.0031621  0.0094483  -0.335    0.738
## I(mean_slope_GSL^2)  0.0008110  0.0006037   1.343    0.179
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -2.062e+01  4.704e+03  -0.004    0.997
## mean_slope_GSL      -2.877e-02  1.614e+02   0.000    1.000
## I(mean_slope_GSL^2)  7.852e-03  1.263e+01   0.001    1.000
```

Session info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
```

```
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Spanish_Spain.utf8  LC_CTYPE=Spanish_Spain.utf8
## [3] LC_MONETARY=Spanish_Spain.utf8 LC_NUMERIC=C
## [5] LC_TIME=Spanish_Spain.utf8
##
## time zone: Europe/Madrid
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] progress_1.2.3    countrycode_1.6.1 furrr_0.3.1       future_1.69.0
##  [5] gridExtra_2.3     readxl_1.4.5      rgbif_3.8.4.5     taxize_0.10.0
##  [9] ggeffects_2.3.2   glmmTMB_1.1.14    sf_1.0-24         lubridate_1.9.5
## [13] forcats_1.0.1     stringr_1.6.0     dplyr_1.2.0       purrr_1.2.1
## [17] tidyr_1.3.2       tibble_3.3.1      ggplot2_4.0.2     tidyverse_2.0.0
## [21] here_1.0.2        readr_2.1.6
##
## loaded via a namespace (and not attached):
##   [1] Rdpack_2.6.5        DBI_1.2.3           sandwich_3.1-1
##   [4] rlang_1.1.7         magrittr_2.0.4      multcomp_1.4-29
##   [7] snakecase_0.11.1    otel_0.2.0          e1071_1.7-17
##  [10] compiler_4.5.2      mgcv_1.9-4          vctrs_0.7.1
##  [13] httpcode_0.3.0      pkgconfig_2.0.3     crayon_1.5.3
##  [16] fastmap_1.2.0       labeling_0.4.3      utf8_1.2.6
##  [19] rmarkdown_2.30      tzdb_0.5.0          haven_2.5.5
##  [22] nloptr_2.2.1        bit_4.6.0           xfun_0.56
##  [25] jsonlite_2.0.0      prettyunits_1.2.0   parallel_4.5.2
##  [28] R6_2.6.1            stringi_1.8.7       RColorBrewer_1.1-3
##  [31] parallelly_1.46.1   boot_1.3-32         cellranger_1.1.0
##  [34] numDeriv_2016.8-1.1 estimability_1.5.1  Rcpp_1.1.1
##  [37] knitr_1.51          triebeard_0.4.1     zoo_1.8-15
##  [40] Matrix_1.7-4        splines_4.5.2       timechange_0.4.0
##  [43] tidyselect_1.2.1    rstudioapi_0.18.0   yaml_2.3.12
##  [46] sjlabelled_1.2.0    TMB_1.9.19          codetools_0.2-20
##  [49] curl_7.0.0          listenv_0.10.0      lattice_0.22-7
##  [52] plyr_1.8.9          withr_3.0.2         S7_0.2.1
##  [55] coda_0.19-4.1       evaluate_1.0.5      survival_3.8-6
##  [58] units_1.0-0         proxy_0.4-29        xml2_1.5.2
##  [61] pillar_1.11.1       whisker_0.4.1       KernSmooth_2.23-26
##  [64] reformulas_0.4.4    insight_1.4.6       generics_0.1.4
##  [67] vroom_1.7.0         rprojroot_2.1.1     hms_1.1.4
##  [70] scales_1.4.0        minqa_1.2.8         globals_0.19.0
##  [73] xtable_1.8-4        class_7.3-23        glue_1.8.0
##  [76] emmeans_2.0.1       lazyeval_0.2.2      tools_4.5.2
##  [79] data.table_1.18.2.1 lme4_1.1-38         mvtnorm_1.3-3
##  [82] grid_4.5.2          ape_5.8-1           urltools_1.7.3.1
```

```
##  [85] rbibutils_2.4.1     datawizard_1.3.0    nlme_3.1-168
##  [88] cli_3.6.5           gtable_0.3.6        oai_0.4.0
##  [91] digest_0.6.39       classInt_0.4-11     crul_1.6.0
##  [94] TH.data_1.1-5       farver_2.1.2        htmltools_0.5.9
##  [97] lifecycle_1.0.5     httr_1.4.7          bit64_4.6.0-1
## [100] MASS_7.3-65
```